



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий (МОСИТ)

КУРСОВАЯ РАБОТА
по дисциплине «Разработка клиент-серверных мобильных приложений»

Тема курсовой работы: «Разработка клиент-серверного мобильного приложения для управления учебными заданиями»

Студент группы ИКБО-63-23 Трисветов Ричард Вунгович


(подпись)

Руководитель
курсовой работы Старший преподаватель
Дворникова Е.М.


(подпись)

Работа представлена к защите «08» 06 2026 г.

Допущен к защите «08» 06 2026 г.

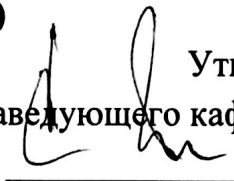
Исавитов, /Исавитов О.А./

Москва 2026 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий (МОСИТ)

Утверждаю
И.о. заведующего кафедрой МОСИТ

Головин С.А.
«16» февраля 2026 г.

ЗАДАНИЕ
на выполнение курсовой работы
по дисциплине «Разработка клиент-серверных мобильных приложений»

Студент Трисветов Ричард Вунгович

Группа ИКБО-63-23

Тема «Разработка клиент-серверного мобильного приложения для управления учебными заданиями»

Исходные данные: Разрабатываемый прототип мобильного приложения должен предоставлять возможности полностью интерактивной системы с понятным дружественным UI и обеспечивать необходимую функциональность, которая формируется в зависимости от заданной темы и предметной области изучаемых вопросов.

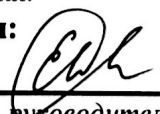
Перечень вопросов, подлежащих разработке, и обязательного графического материала:

1. Описание предметной области мобильного приложения;
2. Сравнительный анализ существующих аналогов мобильного приложения;
3. Обоснование выбора инструментальных средств разработки мобильного приложения;
4. Определение функциональных и нефункциональных требований;
5. Проектирование пользовательского интерфейса мобильного приложения;
6. Логическая модель мобильного приложения (диаграмма в нотации IDEF0, диаграмма классов);
7. Проектирование базы данных мобильного приложения;
8. Разработка клиентской части мобильного приложения;
9. Разработка серверной части мобильного приложения;
10. Тестирование разработанного мобильного приложения.

Отчет по курсовой работе в виде пояснительной записки.

Срок представления к защите курсовой работы:

Задание на курсовую работу выдал


Подпись руководителя

до «30» мая 2026 г.

Дворникова Е.М.

(ФИО руководителя)

«16» февраля 2026 г.

Трисветов Р.В.

(ФИО обучающегося)

«16» февраля 2026 г.

Задание на курсовую работу получил


Подпись обучающегося

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 АНАЛИЗ СОСТОЯНИЯ ОБЪЕКТА ИССЛЕДОВАНИЯ	7
1.1 Описание предметной области мобильного приложения	7
1.2 Сравнительный анализ существующих аналогов	8
1.3 Обоснование выбора инструментальных средств для разработки мобильного приложения	11
2 ПРОЕКТИРОВАНИЕ МОБИЛЬНОГО ПРИЛОЖЕНИЯ	14
2.1 Определение функциональных и нефункциональных требований	14
2.2 Проектирование пользовательского интерфейса	18
2.3 Логическая модель приложения.....	31
2.3.1 Проектирование мобильного приложения.....	31
2.3.2 Проектирование базы данных	37
3 РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ	41
3.1 Разработка клиентской части мобильного приложения	41
3.2 Разработка серверной части мобильного приложения	53
3.2.1 Описание основной логики сервера.....	55
3.2.2 Подключение базы данных к серверу.....	57
3.2.3 Подключение сервера к клиентской части мобильного приложения	59
3.3 Тестирование разработанного приложения	62
ЗАКЛЮЧЕНИЕ.....	64
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	66
ПРИЛОЖЕНИЯ	68
Приложение А. Ссылки на репозитории проекта.....	69
Приложение Б. Скриншоты экранов мобильного приложения	70
Приложение В. Тестовые сценарии мобильного приложения.....	76

ВВЕДЕНИЕ

В настоящее время мобильные приложения являются неотъемлемой частью повседневной деятельности человека. Особенно активно мобильные технологии применяются в образовательной сфере, где они используются для организации учебного процесса, хранения информации, планирования задач и контроля сроков выполнения работ. Современные студенты сталкиваются с большим объемом учебной нагрузки, включающей выполнение домашних заданий, подготовку к лабораторным и практическим работам, контрольным мероприятиям и экзаменам.

Актуальность темы курсовой работы обусловлена необходимостью повышения эффективности организации учебной деятельности с использованием современных клиент-серверных технологий. Многие существующие приложения для планирования задач либо обладают избыточным функционалом, либо не ориентированы непосредственно на учебный процесс. В связи с этим разработка специализированного клиент-серверного мобильного приложения для управления учебными заданиями и дедлайнами является востребованной и практически значимой задачей.

Целью курсовой работы является разработка клиент-серверного мобильного приложения для управления учебными заданиями и дедлайнами, обеспечивающего возможность создания, редактирования и хранения учебных задач, отслеживания сроков их выполнения, а также взаимодействия клиентской части с сервером для хранения и обработки данных.

Для достижения поставленной цели необходимо решить следующие **задачи**:

- выполнить анализ предметной области и существующих аналогов мобильных приложений для планирования задач;
- выбрать инструментальные средства для разработки клиентской и серверной частей приложения;

- определить функциональные и нефункциональные требования к разрабатываемому приложению;
- разработать пользовательский интерфейс мобильного приложения;
- спроектировать архитектуру клиентской и серверной частей приложения;
- разработать структуру базы данных для хранения информации о пользователях, предметах и заданиях;
- реализовать клиентскую часть мобильного приложения на языке Kotlin;
- реализовать серверную часть приложения и настроить взаимодействие клиента с сервером;
- выполнить тестирование разработанного мобильного приложения и проанализировать результаты его работы.

Объектом исследования курсовой работы является процесс организации и контроля выполнения учебных заданий студентами с использованием мобильных технологий.

Предметом исследования является разработка клиент-серверного мобильного приложения для управления учебными заданиями и дедлайнами, включая проектирование пользовательского интерфейса, серверной части, базы данных и механизмов взаимодействия между компонентами системы.

Курсовая работа состоит из введения, трех глав, заключения, списка использованных источников и приложений.

В первой главе рассматривается предметная область разрабатываемого мобильного приложения, проводится анализ существующих аналогов и обосновывается выбор инструментальных средств для разработки клиентской и серверной частей приложения.

Во второй главе определяются функциональные и нефункциональные требования к системе, разрабатывается пользовательский интерфейс приложения, а также выполняется проектирование архитектуры приложения и базы данных.

В третьей главе описывается процесс разработки клиентской и серверной частей мобильного приложения, реализация взаимодействия между ними, а также результаты тестирования разработанного программного продукта.

1 АНАЛИЗ СОСТОЯНИЯ ОБЪЕКТА ИССЛЕДОВАНИЯ

1.1 Описание предметной области мобильного приложения

В настоящее время мобильные приложения для планирования задач и контроля сроков выполнения являются востребованными инструментами как в образовательной сфере, так и в повседневной деятельности пользователей. Современный ритм жизни предполагает необходимость постоянного контроля большого количества задач, учебных заданий, проектов и мероприятий, что делает актуальным использование специализированных цифровых решений для организации времени.

Планирование задач представляет собой процесс распределения дел по приоритетам, срокам выполнения и категориям с целью повышения эффективности деятельности пользователя. Одним из ключевых элементов такого планирования являются дедлайны — установленные временные ограничения, в рамках которых задача должна быть выполнена. Использование дедлайнов позволяет пользователю контролировать сроки выполнения работы, снижать вероятность пропуска важных событий и более рационально распределять нагрузку.

На практике пользователи часто сталкиваются с рядом проблем при организации учебной деятельности. К основным проблемам можно отнести:

- большое количество заданий по различным дисциплинам;
- сложность контроля сроков сдачи;
- отсутствие единого места хранения информации о задачах;
- риск забыть о важных дедлайнах;
- неудобство использования бумажных записей или разрозненных приложений.

Особенно актуальна данная проблема для студентов, которым необходимо одновременно отслеживать выполнение лабораторных работ, практических

заданий, курсовых проектов и других учебных активностей. При отсутствии удобного инструмента планирования снижается эффективность организации учебного процесса и возрастает вероятность нарушения сроков сдачи работ.

Для решения подобных проблем активно используются мобильные приложения, так как мобильные устройства всегда находятся рядом с пользователем и обеспечивают быстрый доступ к информации в любое время. Мобильное приложение позволяет оперативно просматривать список задач, добавлять новые задания, редактировать существующие записи, выполнять поиск по задачам и отслеживать сроки выполнения.

Дополнительным преимуществом мобильных приложений является возможность реализации клиент-серверной архитектуры. В таком случае данные пользователя хранятся на сервере и могут синхронизироваться между устройствами, что повышает надежность хранения информации и обеспечивает возможность дальнейшего расширения функциональности системы.

Таким образом, разработка клиент-серверного мобильного приложения для управления учебными заданиями и дедлайнами является актуальной задачей, направленной на повышение удобства организации учебной деятельности и автоматизацию процессов планирования задач.

1.2 Сравнительный анализ существующих аналогов

В настоящее время существует большое количество мобильных приложений, предназначенных для планирования задач и управления дедлайнами. Для анализа были рассмотрены наиболее популярные решения, обладающие схожим функционалом: Todoist, Trello и Google Tasks.

Todoist

Todoist является одним из наиболее известных приложений для управления задачами и личной продуктивностью. Приложение предоставляет возможность создавать задачи, устанавливать сроки выполнения, формировать категории и использовать систему приоритетов.

К преимуществам Todoist можно отнести:

- удобный и современный интерфейс;
- наличие системы уведомлений;
- поддержку синхронизации между устройствами;
- возможность группировки задач по проектам.

Недостатками приложения являются:

- часть функционала доступна только по платной подписке;
- отсутствие специализации под учебные задачи;
- сложность интерфейса для некоторых пользователей.

Trello

Trello представляет собой систему управления задачами, основанную на использовании досок и карточек. Приложение активно используется как для личного планирования, так и для командной работы.

К преимуществам Trello относятся:

- наглядное представление задач;
- поддержка совместной работы;
- возможность распределения задач по этапам выполнения;
- наличие мобильной и веб-версии.

Недостатками являются:

- избыточность функционала для простого планирования учебных задач;
- необходимость постоянного интернет-соединения для полноценной работы;
- менее удобный поиск по задачам.

Google Tasks

Google Tasks является простым приложением для создания списков задач, интегрированным с сервисами Google.

Преимущества Google Tasks:

- минималистичный интерфейс;
- интеграция с Google Calendar;
- простота использования;
- синхронизация аккаунта между устройствами.

Недостатки приложения:

- ограниченный функционал;
- отсутствие расширенной системы категорий и фильтрации;
- недостаточные возможности персонализации.

Для более наглядного сравнения существующих решений представлена таблица 1.1.

Таблица 1.1 – Сравнение существующих аналогов

Приложение	Уведомления	Синхронизация	Поиск задач	Категории	Авторизация	История поиска	Работа с дедлайнами
Todoist	Да	Да	Да	Да	Да	Да	Да
Trello	Да	Да	Да	Да	Да	Нет	Частично
Google Tasks	Да	Да	Ограничено	Частично	Да	Нет	Да

Проведенный анализ показал, что существующие приложения обладают широкими возможностями для управления задачами, однако имеют ряд ограничений. Некоторые решения перегружены дополнительным функционалом, другие, наоборот, обладают недостаточными возможностями для эффективного управления учебными заданиями и дедлайнами.

Кроме того, многие приложения ориентированы преимущественно на универсальное использование и не учитывают особенности организации учебной деятельности студентов. В связи с этим было принято решение разработать собственное клиент-серверное мобильное приложение, ориентированное на удобное управление учебными задачами, предметами и сроками выполнения заданий.

1.3 Обоснование выбора инструментальных средств для разработки мобильного приложения

Для разработки клиент-серверного мобильного приложения управления учебными заданиями и дедлайнами были выбраны современные программные средства и технологии, обеспечивающие удобство разработки, высокую производительность и возможность дальнейшего расширения функциональности приложения.

В качестве платформы для разработки клиентской части была выбрана операционная система Android, так как она является одной из наиболее распространённых мобильных платформ и предоставляет широкие возможности для создания мобильных приложений различной сложности.

Основным языком программирования был выбран Kotlin. Данный язык официально поддерживается компанией Google и рекомендуется для разработки Android-приложений. Kotlin обладает современным синтаксисом, поддерживает объектно-ориентированное и функциональное программирование, а также обеспечивает высокий уровень безопасности кода.

Для разработки пользовательского интерфейса использовался Jetpack Compose — современный инструмент декларативного построения интерфейсов для Android. Использование Jetpack Compose позволяет уменьшить количество шаблонного кода, упростить создание экранов приложения и ускорить процесс разработки интерфейса.

В качестве архитектуры приложения был выбран клиент-серверный подход. Клиентская часть отвечает за взаимодействие с пользователем, а серверная часть обеспечивает обработку запросов, хранение и передачу данных. Использование клиент-серверной архитектуры позволяет централизованно хранить информацию и обеспечивает возможность синхронизации данных между устройствами.

Для разработки серверной части приложения был выбран фреймворк Ktor. Данный фреймворк предназначен для создания серверных приложений на языке

Kotlin и обеспечивает удобную реализацию REST API, обработку HTTP-запросов и настройку маршрутизации. Использование Ktor позволяет разрабатывать клиентскую и серверную части приложения на одном языке программирования, что упрощает поддержку проекта и повышает удобство разработки.

Взаимодействие между клиентом и сервером реализовано с использованием REST API. Передача данных осуществляется в формате JSON посредством HTTP-запросов. Использование REST API обеспечивает независимость клиентской и серверной частей приложения и упрощает дальнейшее расширение функциональности системы.

Для хранения данных была выбрана система управления базами данных PostgreSQL. PostgreSQL является современной реляционной системой управления базами данных, обеспечивающей высокую производительность, надёжность хранения информации и поддержку сложных запросов.

Для авторизации и аутентификации пользователей применяется технология JWT (JSON Web Token). После успешного входа в систему сервер генерирует токен, который используется клиентским приложением для выполнения защищённых запросов. Использование JWT позволяет безопасно организовать взаимодействие между клиентом и сервером.

Для выполнения сетевых запросов в Android-приложении используется библиотека Retrofit. Данная библиотека обеспечивает взаимодействие клиентской части приложения с REST API сервера, реализованного с использованием фреймворка Ktor, а также позволяет автоматически преобразовывать JSON-ответы в объекты Kotlin.

Дополнительно в приложении используется библиотека Room для локального хранения данных на устройстве пользователя. Использование Room позволяет ускорить доступ к информации и обеспечить сохранение отдельных данных при временном отсутствии интернет-соединения.

Таким образом, выбранный набор инструментальных средств обеспечивает возможность создания современного клиент-серверного

мобильного приложения, обладающего удобным пользовательским интерфейсом, высокой производительностью и возможностью дальнейшего развития функциональности системы.

Таким образом, в первой главе была рассмотрена предметная область разрабатываемого приложения, выполнен анализ существующих аналогов и обоснован выбор инструментальных средств. Проведённый анализ показал актуальность разработки клиент-серверного мобильного приложения для управления учебными заданиями и дедлайнами.

2 ПРОЕКТИРОВАНИЕ МОБИЛЬНОГО ПРИЛОЖЕНИЯ

Проектирование мобильного приложения является важным этапом разработки, так как на данном этапе определяются основные требования к системе, структура пользовательского интерфейса, логика взаимодействия клиента и сервера, а также модель данных.

Разрабатываемое мобильное приложение предназначено для управления учебными заданиями и дедлайнами. Пользователь приложения может регистрироваться, выполнять вход в систему, создавать учебные предметы, добавлять задания, указывать сроки выполнения, просматривать список задач, выполнять поиск записей.

В рамках проектирования были определены функциональные и нефункциональные требования, разработана логика пользовательского взаимодействия, спроектированы основные экраны приложения, а также определена структура данных, используемая в клиентской и серверной частях системы.

2.1 Определение функциональных и нефункциональных требований

Требования к мобильному приложению определяют, какие функции должна выполнять система и каким качественным характеристикам она должна соответствовать. В рамках разрабатываемого приложения требования были разделены на функциональные и нефункциональные.

Функциональные требования описывают основные возможности приложения, доступные пользователю. К функциональным требованиям разрабатываемого мобильного приложения относятся:

- регистрация нового пользователя;
- авторизация пользователя в системе;

- выход из учетной записи;
- создание учебного предмета;
- просмотр списка учебных предметов;
- редактирование учебного предмета;
- удаление учебного предмета;
- создание учебной задачи;
- просмотр списка задач;
- редактирование задачи;
- удаление задачи;
- указание срока выполнения задачи;
- отображение задач, связанных с выбранным предметом;
- поиск задач по введенному запросу;
- очистка поискового запроса;
- сохранение истории поиска;
- отображение состояния загрузки данных;
- отображение сообщений об ошибках при неудачном запросе;
- повторная отправка запроса при ошибке;
- переключение светлой и темной темы приложения.

Нефункциональные требования описывают качественные характеристики приложения, которые не связаны напрямую с отдельными функциями, но влияют на удобство, надежность и корректность работы системы. К нефункциональным требованиям относятся:

- интерфейс приложения должен быть реализован на русском языке;
- приложение должно иметь понятную структуру экранов и навигации;
- пользовательский интерфейс должен соответствовать разработанному макету;
- приложение должно корректно отображаться на мобильных устройствах с разными размерами экранов;
- данные пользователя должны быть изолированы от данных других пользователей;

- взаимодействие клиентской части с сервером должно выполняться через API;
- серверная часть должна обрабатывать запросы клиента и возвращать корректные ответы;
- база данных должна обеспечивать хранение информации о пользователях, предметах и задачах;
- приложение должно сохранять введенный поисковый запрос при изменении конфигурации экрана;
- при выполнении сетевых запросов пользователю должен отображаться индикатор загрузки;
- при отсутствии данных или ошибке должны отображаться соответствующие информационные сообщения;
- приложение должно обеспечивать стабильную работу основных функций без аварийного завершения.

Таким образом, на основе анализа предметной области были определены основные требования к разрабатываемому клиент-серверному мобильному приложению. Эти требования используются далее при проектировании пользовательского интерфейса, логической модели приложения, структуры базы данных и программной реализации системы.



Рисунок 2.1 — Use case диаграмма мобильного приложения

На рисунке 2.1 представлена use case диаграмма разрабатываемого мобильного приложения. Диаграмма вариантов использования отражает основные возможности системы и взаимодействие пользователя с приложением.

Основным актером системы является пользователь, который взаимодействует с мобильным приложением для выполнения операций, связанных с управлением учебными предметами и задачами. Пользователь может выполнять регистрацию и авторизацию, создавать и редактировать предметы, управлять задачами, осуществлять поиск необходимых записей и изменять настройки интерфейса приложения.

Диаграмма вариантов использования позволяет наглядно представить функциональные возможности системы и определить основные сценарии взаимодействия пользователя с приложением. На основе данной диаграммы далее выполняется проектирование пользовательского интерфейса, логической модели приложения и структуры базы данных.

2.2 Проектирование пользовательского интерфейса

Проектирование пользовательского интерфейса является важным этапом разработки мобильного приложения, так как от качества интерфейса зависит удобство взаимодействия пользователя с системой. Основной задачей при проектировании интерфейса являлось создание простого, понятного и современного пользовательского интерфейса, обеспечивающего быстрый доступ к основным функциям приложения.

Для разработки макетов экранов использовался графический редактор Figma. С помощью Figma были спроектированы основные экраны мобильного приложения, определены расположение элементов интерфейса, структура навигации и межэкранное взаимодействие.

При проектировании интерфейса учитывались следующие требования:

- минимальное количество действий для выполнения основных операций;
- логичное расположение элементов интерфейса;

- единый стиль оформления экранов;
- поддержка светлой и темной темы;
- удобная навигация между разделами приложения;
- адаптация интерфейса под мобильные устройства.

В мобильном приложении были реализованы основные экраны, обеспечивающие выполнение пользовательских сценариев работы с системой. Каждый экран отвечает за отдельную функциональную часть приложения и обеспечивает взаимодействие пользователя с серверной частью системы через API.

В процессе проектирования были разработаны следующие экраны:

- экран авторизации;
- экран регистрации;
- главный экран приложения с задачами на сегодня;
- экран создания задачи;
- экран редактирования задачи;
- экран задач на неделю;
- экран предметов;
- экран создания предмета;
- экран редактирования предмета;
- экран задач выбранного предмета;
- экран поиска;
- экран настроек.

Экран авторизации

Экран авторизации предназначен для входа пользователя в систему. На данном экране пользователь вводит адрес электронной почты и пароль, после чего выполняется проверка введенных данных на сервере.

После успешной авторизации пользователь получает доступ к основному функционалу мобильного приложения. В случае ошибки авторизации отображается соответствующее сообщение об ошибке. Также на экране

предусмотрена возможность перехода к экрану регистрации нового пользователя.

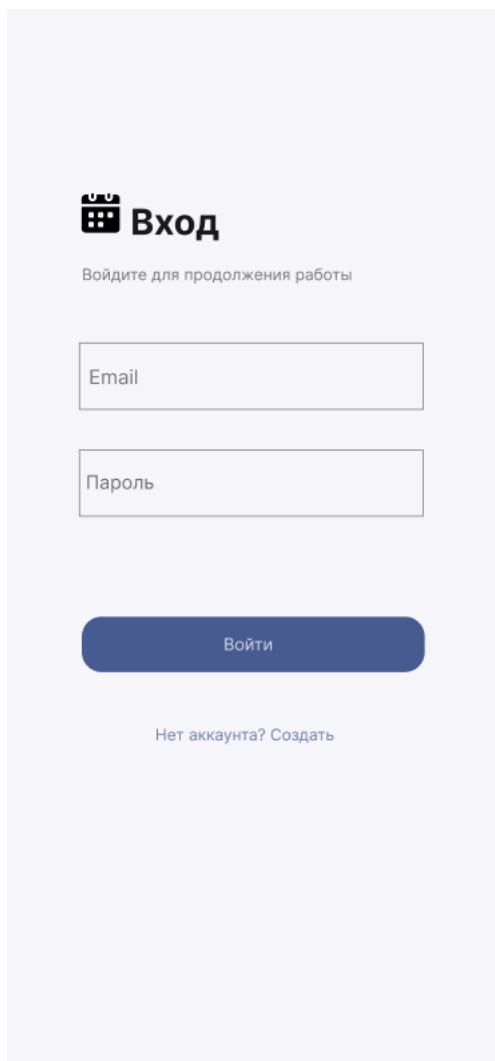
The image shows a mobile application login screen. At the top, there is a black icon of a calendar or document with a checkmark, followed by the word "Вход" in bold. Below this is the instruction "Войдите для продолжения работы". There are two input fields: the first is labeled "Email" and the second is labeled "Пароль". Below the password field is a blue button with the text "Войти". At the bottom, there is a link that says "Нет аккаунта? Создать".

Рисунок 2.2 – Экран авторизации

Экран регистрации

Экран регистрации предназначен для создания новой учетной записи пользователя. Пользователь вводит адрес электронной почты, пароль и подтверждает регистрацию в системе.

После успешной регистрации данные пользователя сохраняются на сервере, а пользователь получает возможность авторизоваться в приложении и использовать его функциональные возможности.

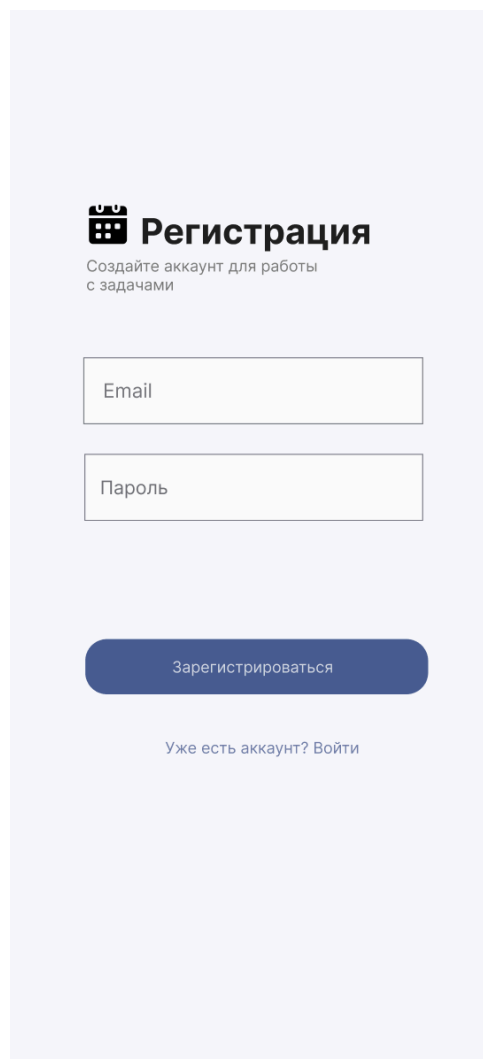


Рисунок 2.3 – Экран регистрации

Главный экран приложения

Главный экран приложения предназначен для отображения актуальных учебных задач пользователя. На экране отображаются задачи, запланированные на текущий день, а также просроченные задачи, требующие внимания пользователя.

Главный экран обеспечивает быстрый доступ к основной информации о текущих дедлайнах и позволяет пользователю своевременно отслеживать выполнение учебных заданий. Также с главного экрана пользователь может перейти к редактированию выбранной задачи, созданию новой задачи, просмотру задач на неделю, экрану предметов, поиску и настройкам приложения.

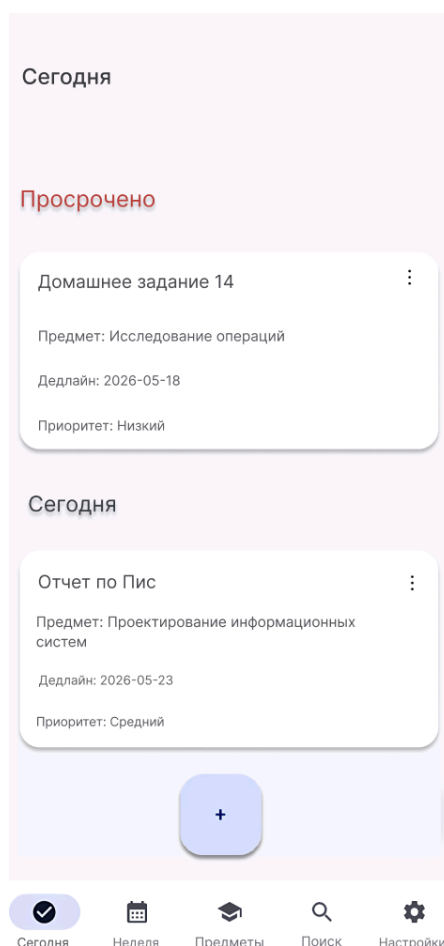


Рисунок 2.4 – Главный экран приложения

Экран редактирования задачи

Экран редактирования задачи предназначен для создания и изменения информации о задаче. Пользователь может указать название задачи, описание, предмет, срок выполнения задания и другие параметры.

После сохранения изменения передаются на сервер и обновляются в базе данных приложения. Экран обеспечивает удобное управление учебными задачами и позволяет пользователю изменять параметры задания в процессе работы с приложением.

Назад

Создание/ Редактирование задачи

Название задачи

Описание

Предмет

Дедлайн

Статус

☒ В планах

☐ В работе

☐ Выполнено

☐ Просрочено

Приоритет

☒ Низкий

☐ Средний

☐ Высокий

Рисунок 2.5 – Экран редактирования задачи

Экран задач на неделю

Экран задач на неделю предназначен для отображения задач пользователя в рамках недельного расписания. На экране отображаются задания с учетом сроков выполнения, что позволяет пользователю планировать учебную деятельность на ближайшие дни.

Использование недельного представления задач повышает удобство планирования и помогает пользователю контролировать распределение учебной нагрузки.

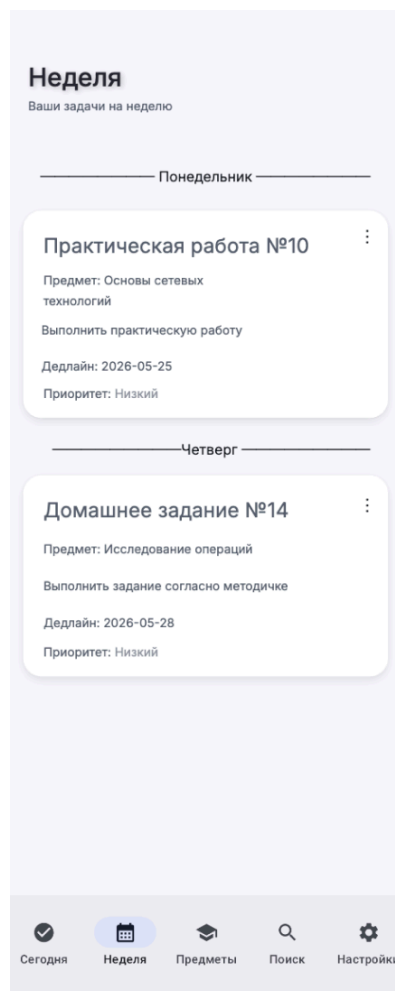


Рисунок 2.6 – Экран задач на неделю

Экран предметов

Экран предметов предназначен для управления учебными предметами пользователя. На данном экране отображается список созданных предметов, а также реализована возможность добавления, изменения и удаления предметов.

Использование предметов позволяет структурировать задачи и упрощает организацию учебной деятельности пользователя.

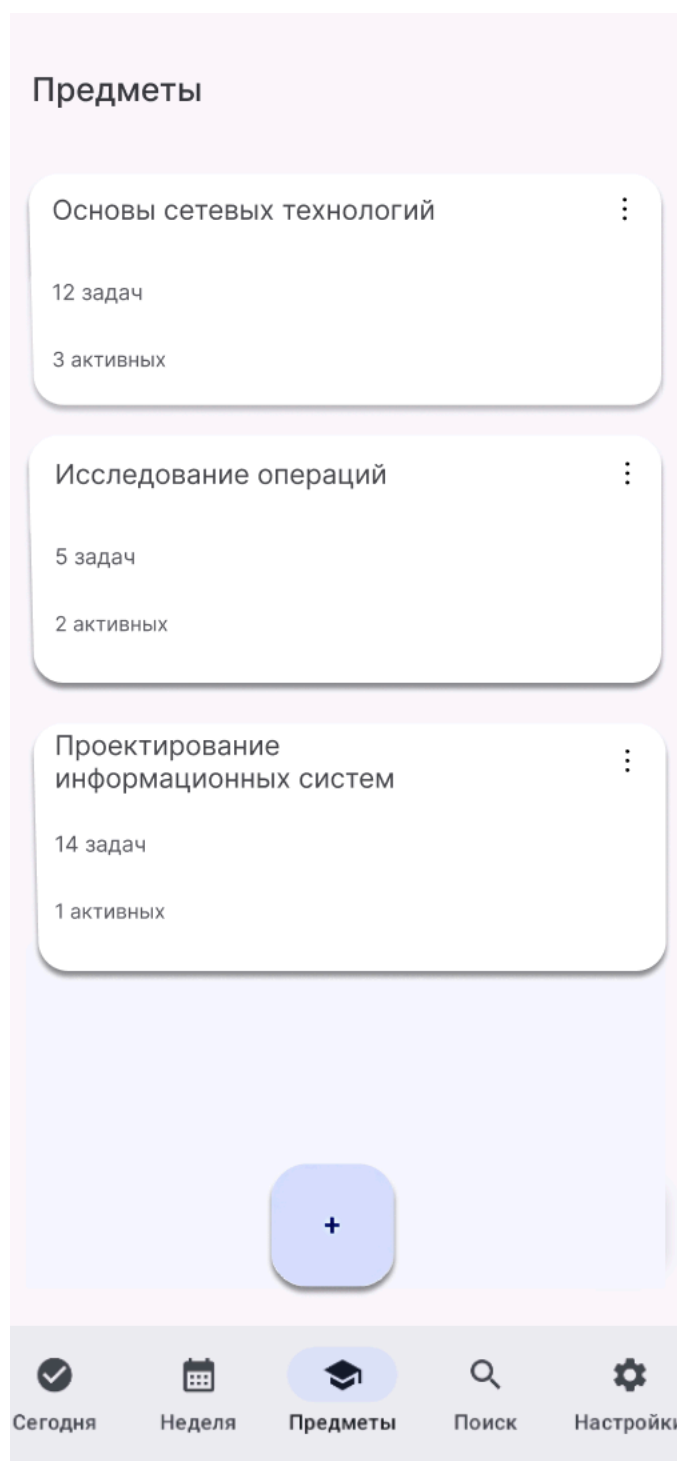


Рисунок 2.7 – Экран предметов

Экран задач выбранного предмета

Экран задач выбранного предмета предназначен для отображения всех задач, относящихся к конкретному учебному предмету. Пользователь может просматривать список заданий, связанных с выбранной дисциплиной, отслеживать сроки выполнения задач, а также изменять статус выполнения заданий.

На экране реализована возможность перехода к редактированию выбранной задачи, а также удаления или отметки задачи как выполненной. Использование данного экрана позволяет структурировать учебные задания по предметам и упрощает управление учебным процессом.

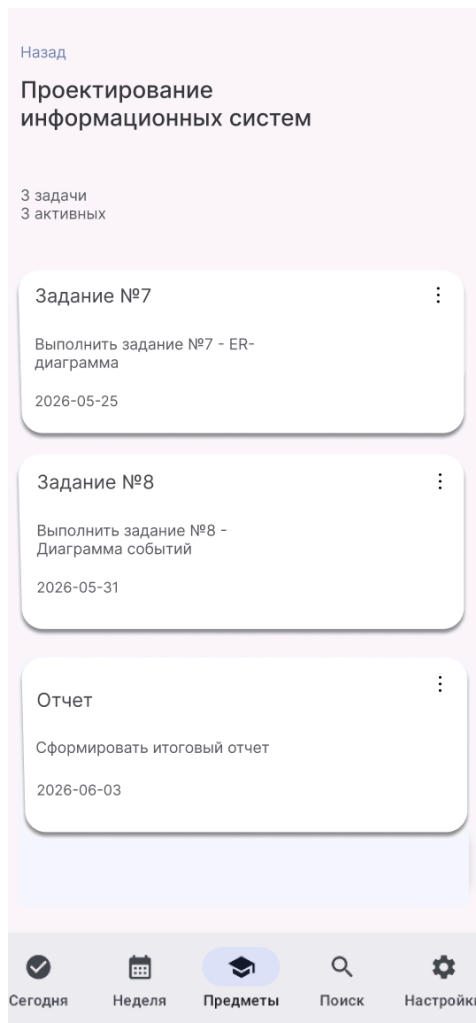


Рисунок 2.8 – Экран задач выбранного предмета

Окно редактирования предмета

Окно редактирования предмета предназначено для создания нового предмета или изменения информации о существующем предмете. Пользователь может указать название предмета и сохранить изменения.

После подтверждения данные обновляются на сервере и становятся доступны в приложении.

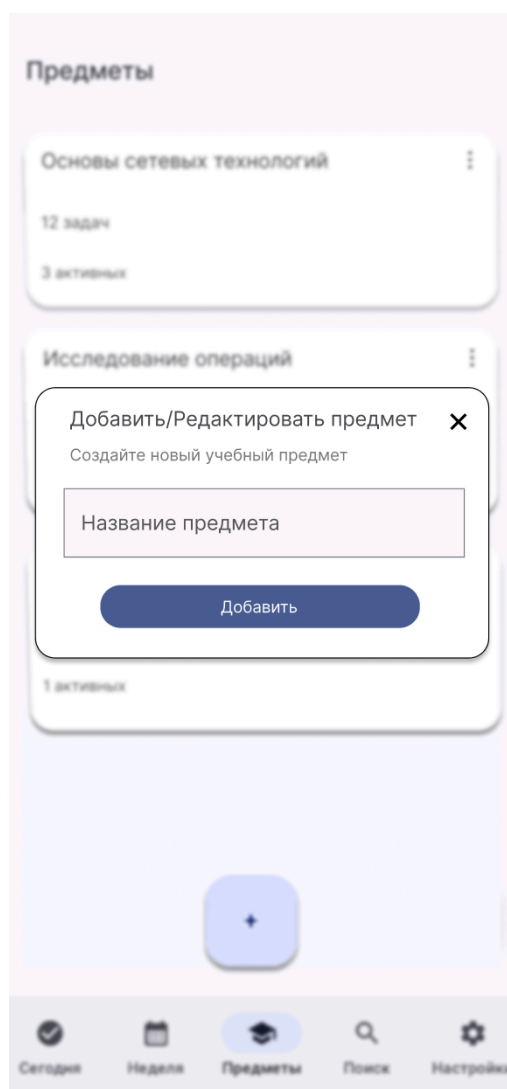


Рисунок 2.9 – Окно редактирования предмета

Экран поиска

Экран поиска предназначен для поиска задач по введенному запросу. Пользователь может вводить текст поискового запроса, просматривать результаты поиска и использовать историю ранее введенных запросов.

На экране реализована кнопка очистки поискового запроса, отображение индикатора загрузки, а также обработка ситуаций отсутствия результатов или ошибок выполнения запроса.

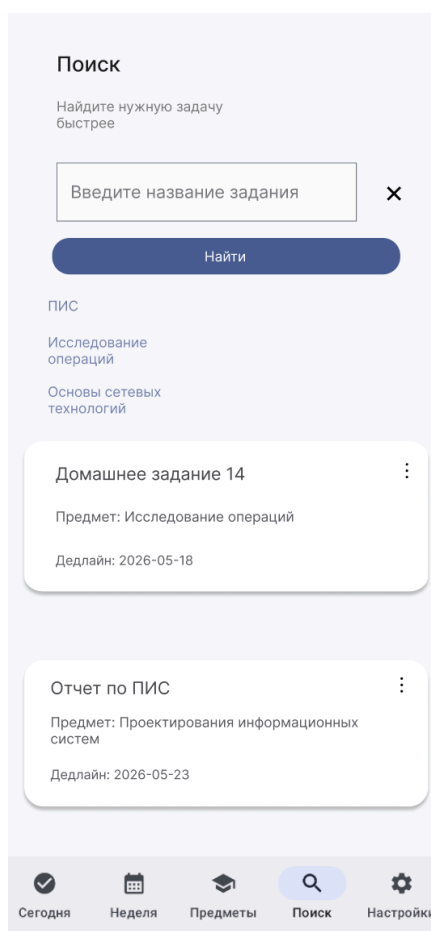


Рисунок 2.10 – Экран поиска

Экран настроек

Экран настроек предназначен для изменения параметров приложения и управления учетной записью пользователя. На данном экране реализована возможность переключения светлой и темной темы интерфейса, а также функция выхода из учетной записи пользователя.

Использование экрана настроек позволяет пользователю персонализировать внешний вид приложения и управлять параметрами авторизации.

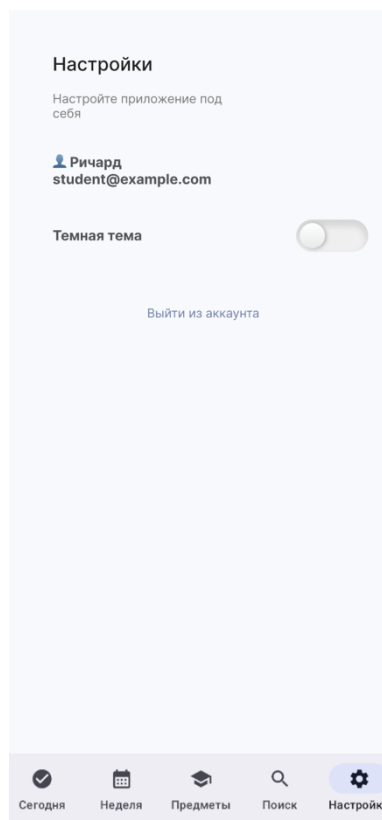


Рисунок 2.11 – Экран настроек

Для обеспечения удобной навигации в мобильном приложении было реализовано межэкранное взаимодействие между основными разделами системы. Переходы между экранами позволяют пользователю быстро получать доступ к необходимым функциям приложения и выполнять основные сценарии работы.

После успешной авторизации пользователь переходит на главный экран приложения, на котором отображаются актуальные задачи и просроченные задания. С главного экрана пользователь может перейти к экрану задач на неделю, экрану предметов, поиску и настройкам приложения с помощью нижнего меню навигации.

Экран предметов обеспечивает переход к списку задач выбранного предмета, а также к окну создания или редактирования предмета. Экран задач выбранного предмета позволяет пользователю редактировать задачи, изменять их статус или удалять задания.

Экран поиска предоставляет возможность поиска задач по текстовому запросу и отображает историю поиска пользователя. Экран настроек

предназначен для изменения параметров интерфейса приложения и выхода из учетной записи.

Таким образом, реализованная структура навигации обеспечивает логичное и удобное взаимодействие пользователя с мобильным приложением.

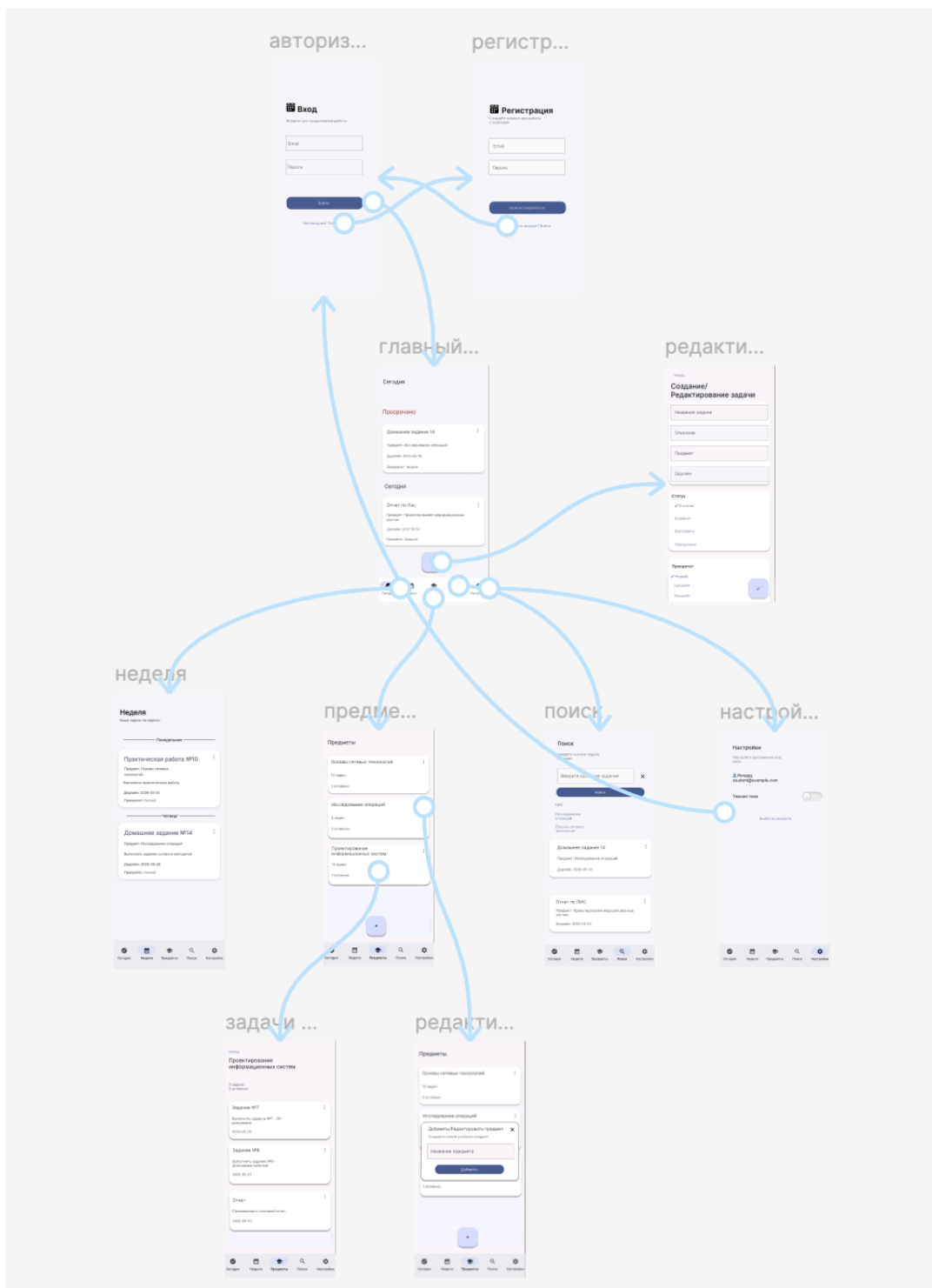


Рисунок 2.12 - Схема межэкранного взаимодействия

2.3 Логическая модель приложения

Логическая модель приложения определяет структуру взаимодействия компонентов системы, основные сущности приложения, а также способы хранения и обработки данных. Проектирование логической модели позволяет определить архитектуру мобильного приложения, взаимосвязи между его компонентами и структуру базы данных.

В рамках проектирования были определены основные сущности системы, структура клиентской и серверной частей приложения, а также механизмы взаимодействия между ними. Для описания архитектуры приложения использовались диаграмма IDEF0, диаграмма классов и логическая модель базы данных.

2.3.1 Проектирование мобильного приложения

Проектирование мобильного приложения включает определение основных компонентов системы, описание потоков данных и взаимодействия между клиентской и серверной частями приложения.

Разрабатываемое приложение построено по клиент-серверной архитектуре. Клиентская часть реализована в виде Android-приложения на языке Kotlin с использованием Jetpack Compose. Серверная часть разработана с использованием фреймворка Ktor и обеспечивает обработку запросов клиента, взаимодействие с базой данных и выполнение бизнес-логики приложения.

Обмен данными между клиентом и сервером осуществляется посредством HTTP-запросов через REST API. Для передачи данных используется формат JSON. Хранение данных осуществляется в базе данных PostgreSQL.

В приложении реализованы основные функции управления учебными предметами и задачами, авторизация пользователей, поиск задач и хранение пользовательских данных.

Для описания функциональной структуры мобильного приложения была использована методология IDEF0. Диаграммы IDEF0 позволяют наглядно представить основные процессы системы, входные и выходные данные, механизмы выполнения процессов, а также управляющие воздействия.

На контекстной диаграмме верхнего уровня представлена общая функциональная модель мобильного приложения для управления учебными заданиями и дедлайнами.

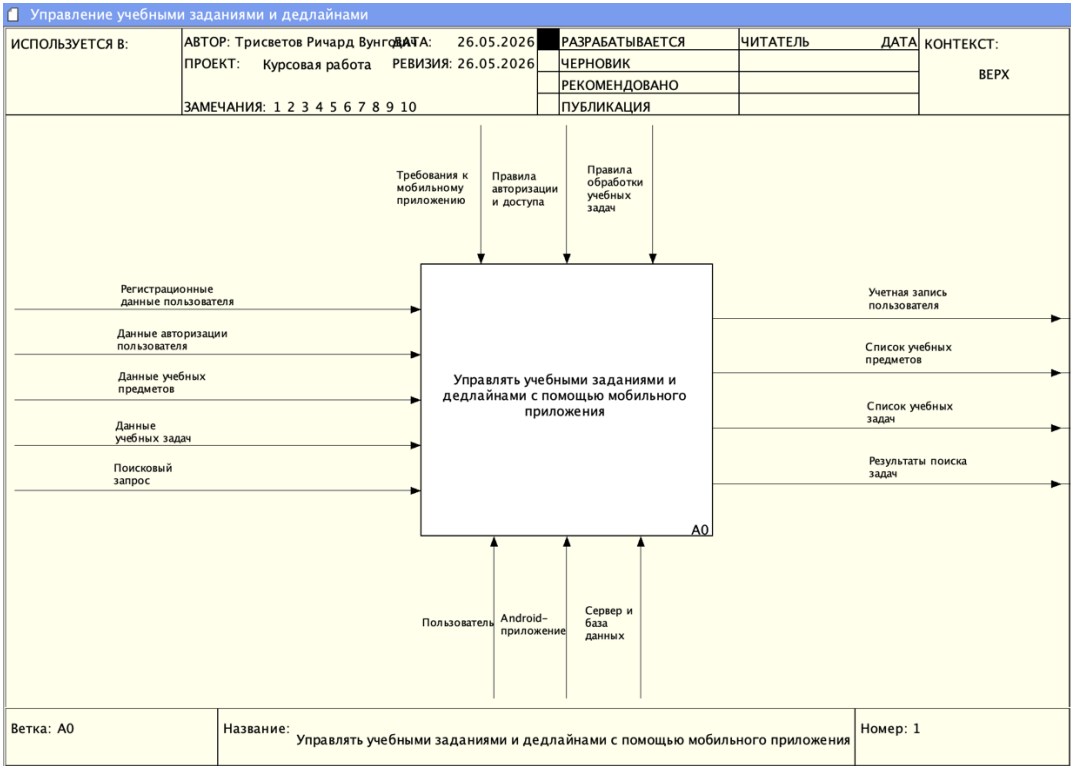


Рисунок 2.13 — Контекстная IDEF0-диаграмма мобильного приложения

На контекстной диаграмме представлены основные входные данные системы, включая данные пользователя, информацию о предметах и задачах, а также поисковые запросы. В качестве механизмов выполнения процессов выступают пользователь, мобильное Android-приложение и серверная часть системы с базой данных.

В результате работы системы формируются учетные данные пользователя, список учебных предметов, список задач и результаты поиска. Управляющее воздействие на систему оказывают требования к мобильному приложению, правила авторизации и правила обработки учебных задач.

Для более детального описания функциональной структуры приложения была выполнена декомпозиция основного процесса на несколько взаимосвязанных подпроцессов.

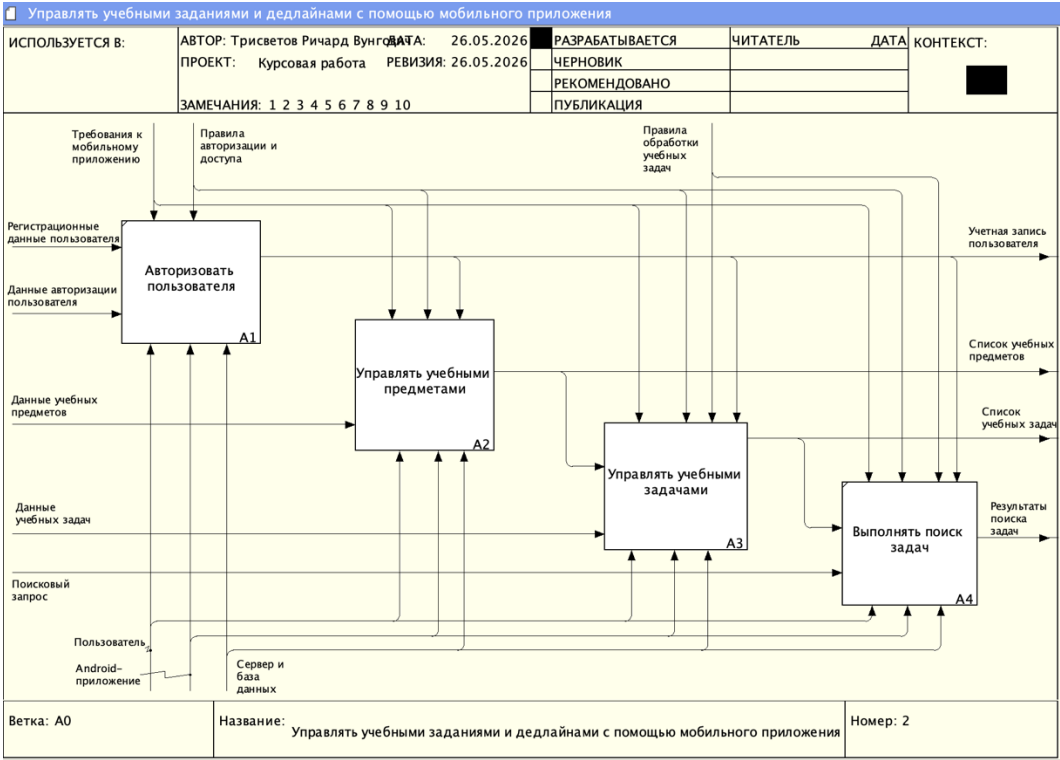


Рисунок 2.14 — Декомпозиция IDEF0-диаграммы мобильного приложения

На диаграмме декомпозиции представлены основные функциональные процессы мобильного приложения: авторизация пользователя, управление учебными предметами, управление учебными задачами и выполнение поиска задач.

Процесс авторизации обеспечивает регистрацию и вход пользователя в систему. Процесс управления предметами отвечает за создание, изменение, просмотр и удаление учебных предметов. Процесс управления задачами обеспечивает работу с учебными заданиями и дедлайнами. Процесс поиска предназначен для поиска задач по пользовательскому запросу.

Наиболее важным процессом системы является управление учебными задачами, поэтому для него была выполнена дополнительная декомпозиция.

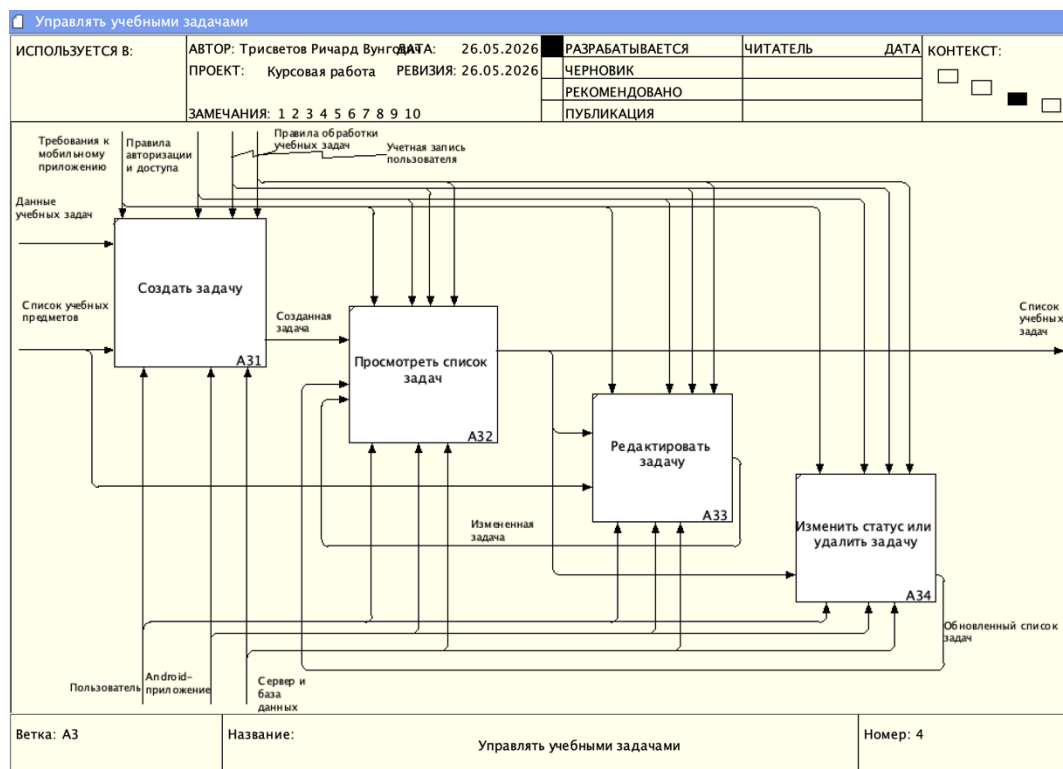


Рисунок 2.15 — Декомпозиция процесса управления учебными задачами

Диаграмма управления учебными задачами отражает основные операции по работе с заданиями: создание задачи, просмотр списка задач, редактирование задач, а также изменение статуса и удаление заданий.

Процесс создания задачи обеспечивает добавление нового задания в систему. Просмотр списка задач предназначен для отображения актуальных задач пользователя. Редактирование задачи позволяет изменять параметры задания, а изменение статуса и удаление задачи обеспечивает управление состоянием учебных заданий.

Для описания процесса управления учебными предметами также была выполнена декомпозиция.

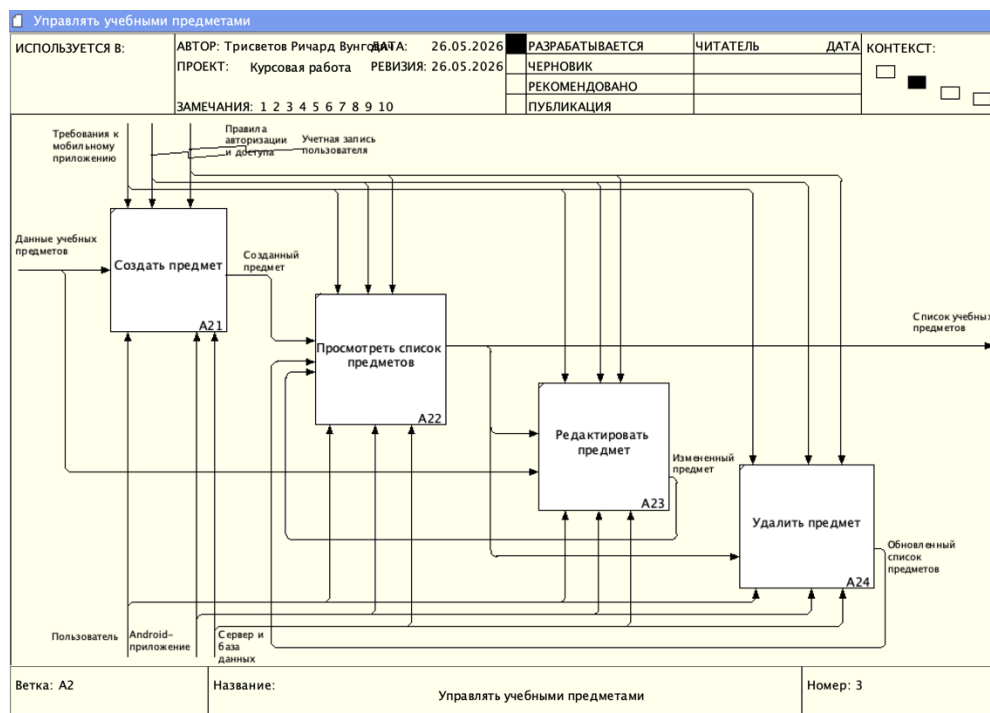


Рисунок 2.16 — Декомпозиция процесса управления учебными предметами

Диаграмма управления учебными предметами отражает основные операции по работе с предметами: создание предмета, просмотр списка предметов, редактирование и удаление предметов.

Использование учебных предметов позволяет структурировать задачи пользователя и распределять задания по отдельным дисциплинам. Реализованная функциональная модель приложения обеспечивает логичную организацию процессов и взаимодействие между основными компонентами системы.

Для описания объектной структуры мобильного приложения была разработана UML-диаграмма классов. Диаграмма классов позволяет определить основные сущности системы, их атрибуты, методы и взаимосвязи между объектами приложения.

В диаграмме были выделены классы, отвечающие за хранение данных пользователей, учебных предметов и задач, а также сервисные классы, реализующие основную бизнес-логику мобильного приложения.

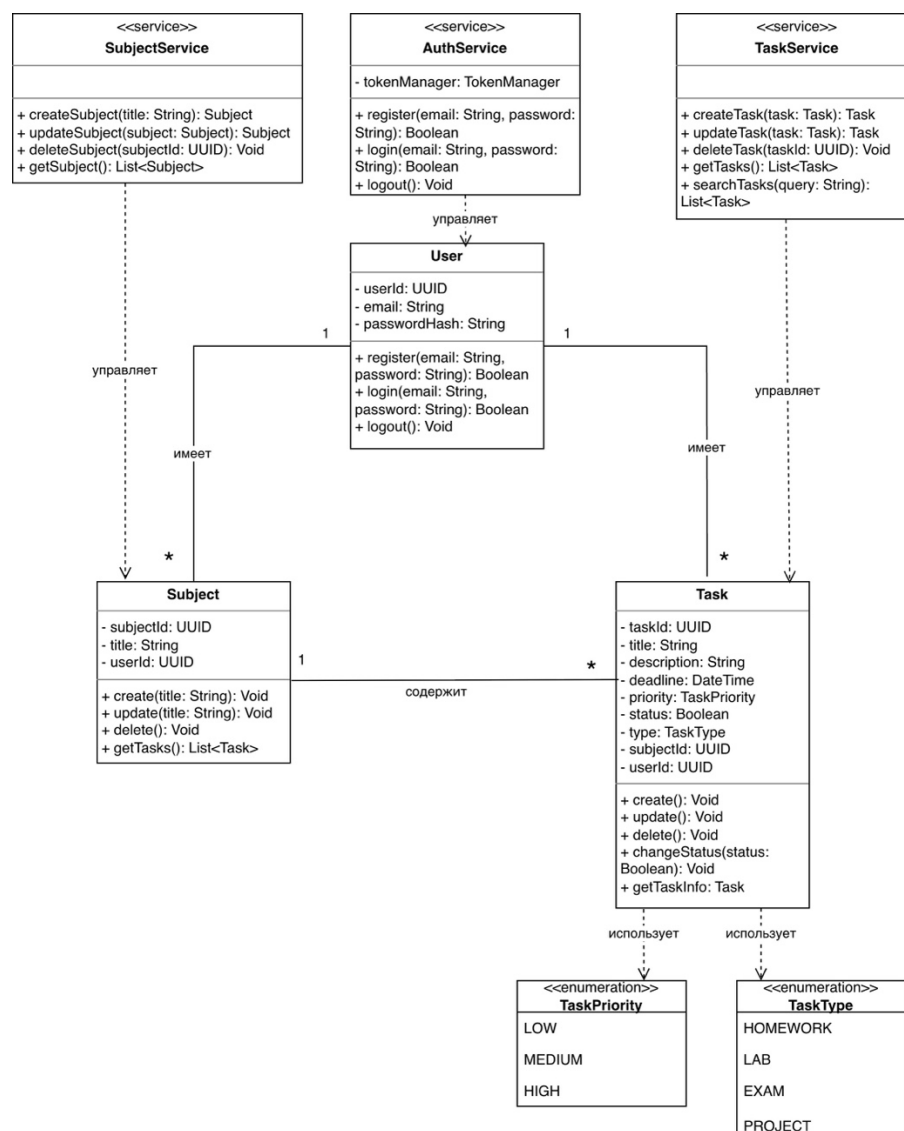


Рисунок 2.17 — UML-диаграмма классов мобильного приложения

Класс User предназначен для хранения информации о пользователях системы и содержит данные, необходимые для регистрации и авторизации пользователя.

Класс Subject описывает учебные предметы пользователя и обеспечивает хранение информации о дисциплинах, используемых при организации учебных задач.

Класс Task предназначен для хранения информации об учебных заданиях. Класс содержит данные о названии задачи, описании, сроке выполнения, приоритете, типе и статусе выполнения задания.

Сервисные классы AuthService, SubjectService и TaskService реализуют основную бизнес-логику мобильного приложения. Данные классы обеспечивают

выполнение операций регистрации и авторизации пользователей, управление учебными предметами и обработку учебных задач.

Дополнительно в диаграмме были выделены перечисления TaskPriority и TaskType, используемые для определения приоритета и типа учебной задачи.

Связи между классами отражают взаимодействие объектов системы и структуру мобильного приложения. Один пользователь может иметь несколько учебных предметов и задач, а один учебный предмет может содержать несколько связанных задач.

Разработанная UML-диаграмма классов отражает логическую структуру мобильного приложения и используется при проектировании базы данных и реализации серверной части системы.

2.3.2 Проектирование базы данных

База данных является важной частью клиент-серверного мобильного приложения и предназначена для хранения информации о пользователях, учебных предметах и задачах. Проектирование базы данных позволяет определить структуру хранения данных, взаимосвязи между сущностями и обеспечить корректное взаимодействие серверной части приложения с хранилищем данных.

Для хранения данных в разрабатываемом приложении используется система управления базами данных PostgreSQL. В базе данных реализовано хранение учетных записей пользователей, учебных предметов и учебных задач.

Логическая модель базы данных была разработана с учетом структуры мобильного приложения и основных пользовательских сценариев работы с системой.

На основе выделенных сущностей была разработана логическая модель базы данных мобильного приложения. Модель базы данных отражает структуру хранения информации и взаимосвязи между основными таблицами системы.

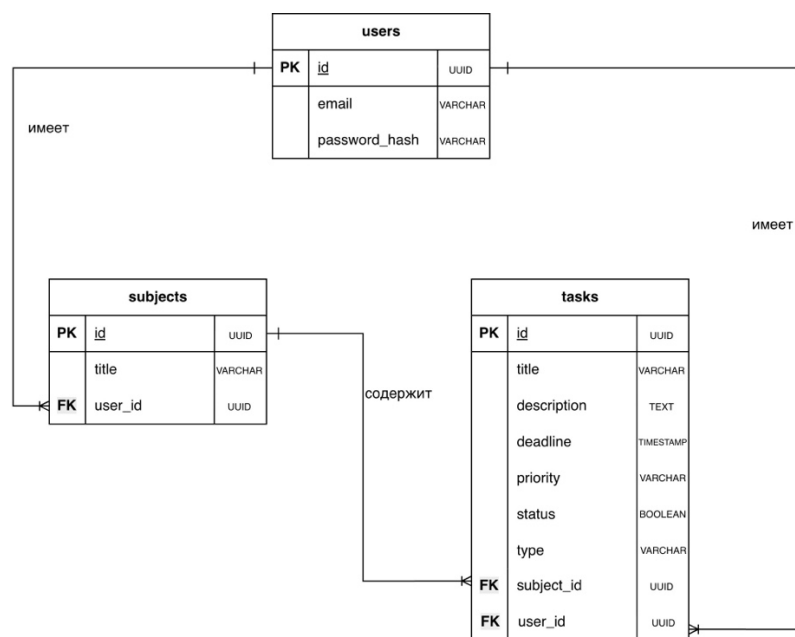


Рисунок 2.18 — Логическая модель базы данных мобильного приложения

В базе данных были реализованы таблицы `users`, `subjects` и `tasks`. Таблица `users` предназначена для хранения информации о пользователях приложения и содержит данные, необходимые для регистрации и авторизации.

Таблица `subjects` содержит информацию об учебных предметах пользователя. Каждый предмет связан с определенным пользователем посредством внешнего ключа `user_id`.

Таблица `tasks` предназначена для хранения информации об учебных задачах. Таблица содержит название задачи, описание, срок выполнения, приоритет, статус выполнения и тип задания. Каждая задача связана с пользователем и учебным предметом с помощью внешних ключей `user_id` и `subject_id`.

Связи между таблицами обеспечивают структурированное хранение данных и позволяют реализовать корректное взаимодействие между пользователями, предметами и задачами мобильного приложения.

Для более подробного описания структуры базы данных были определены основные поля таблиц, их типы данных и назначение.

Структура таблицы `users` представлена в таблице 2.1.

Таблица 2.1 — Структура таблицы users

Поле	Тип данных	Назначение
id	UUID	Уникальный идентификатор пользователя
email	VARCHAR	Адрес электронной почты пользователя
password_hash	VARCHAR	Хэш пароля пользователя

Структура таблицы subjects представлена в таблице 2.2.

Таблица 2.2 — Структура таблицы subjects

Поле	Тип данных	Назначение
id	UUID	Уникальный идентификатор предмета
title	VARCHAR	Название учебного предмета
user_id	UUID	Идентификатор пользователя, которому принадлежит предмет

Структура таблицы tasks представлена в таблице 2.3.

Таблица 2.3 — Структура таблицы tasks

Поле	Тип данных	Назначение
id	UUID	Уникальный идентификатор задачи
title	VARCHAR	Название задачи
description	TEXT	Описание задачи
deadline	TIMESTAMP	Срок выполнения задачи
priority	VARCHAR	Приоритет задачи
status	BOOLEAN	Статус выполнения задачи
type	VARCHAR	Тип учебного задания
subject_id	UUID	Идентификатор предмета
user_id	UUID	Идентификатор пользователя

Разработанная структура базы данных обеспечивает хранение информации о пользователях, учебных предметах и задачах, а также поддерживает

взаимосвязи между сущностями системы. Использование внешних ключей позволяет обеспечить целостность данных и корректную связь задач с предметами и пользователями.

Спроектированная база данных используется серверной частью мобильного приложения для хранения, получения и обновления информации в процессе работы системы.

3 РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ

В ходе выполнения курсовой работы было разработано клиент-серверное мобильное приложение для управления учебными заданиями и дедлайнами. Приложение предназначено для хранения учебных предметов, создания задач, отслеживания сроков выполнения, поиска нужных заданий и планирования учебной деятельности пользователя.

Разработка приложения включала создание клиентской части для операционной системы Android, реализацию серверной части, организацию взаимодействия между клиентом и сервером, а также проверку работоспособности основных функций приложения. Клиентская часть была реализована на языке Kotlin с использованием Jetpack Compose, а серверная часть — с использованием фреймворка Ktor. Для обмена данными между клиентом и сервером применялся REST API, а передача данных выполнялась в формате JSON.

3.1 Разработка клиентской части мобильного приложения

Клиентская часть мобильного приложения была разработана для операционной системы Android. Основной задачей клиентской части является предоставление пользователю удобного интерфейса для работы с учебными предметами, задачами и дедлайнами. Интерфейс приложения реализован на русском языке и соответствует макетам, разработанным на этапе проектирования.

Для построения пользовательского интерфейса использовался Jetpack Compose. Данный инструмент позволяет описывать интерфейс декларативным способом, что упрощает разработку экранов, управление состояниями и обновление элементов интерфейса при изменении данных. В приложении были реализованы экраны регистрации, авторизации, главного списка задач, списка задач на неделю, списка учебных предметов, задач выбранного предмета, поиска, настроек, а также окна добавления и редактирования данных.

После запуска приложения пользователь попадает на экран авторизации. Если учетная запись отсутствует, пользователь может перейти на экран регистрации и создать новый аккаунт. После успешной авторизации открывается основная часть приложения, где доступны функции просмотра задач, управления предметами и поиска информации.

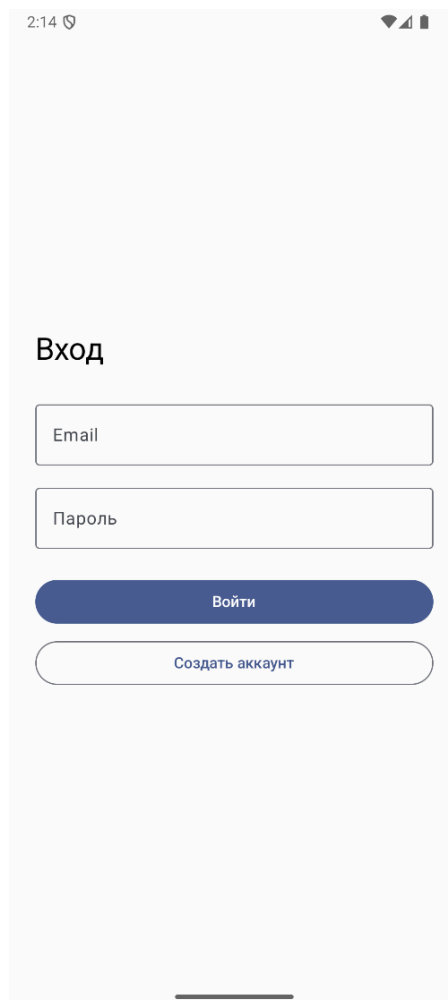


Рисунок 3.1 — Экран авторизации пользователя

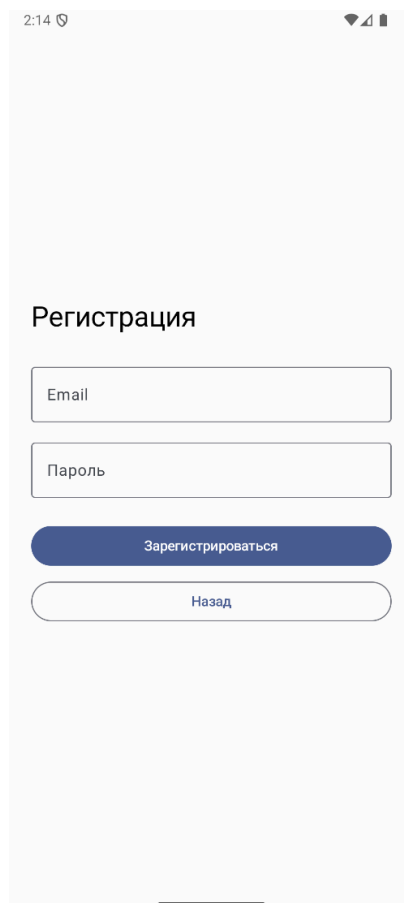


Рисунок 3.2 — Экран регистрации нового пользователя

Основной экран приложения предназначен для отображения учебных задач. На нем пользователь может просматривать список заданий, видеть их название, связанный учебный предмет, дату дедлайна и текущий статус выполнения. Для удобства работы задачи отображаются в виде карточек, что делает интерфейс более наглядным.

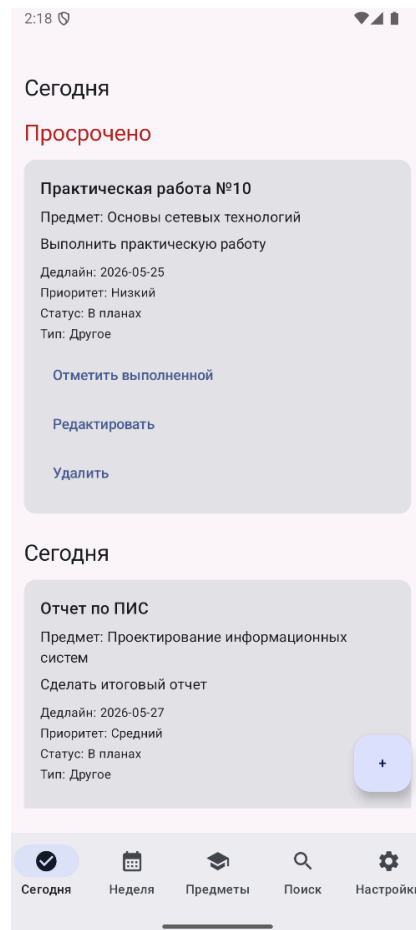


Рисунок 3.3 — Главный экран приложения со списком задач

Экран учебных предметов предназначен для отображения всех предметов, добавленных пользователем. Для каждого предмета отображается его название. Пользователь может перейти к просмотру задач выбранного предмета, а также добавить новый предмет или изменить существующий. Для отображения списка предметов использовался компонент LazyColumn, обеспечивающий эффективную работу со списками данных.

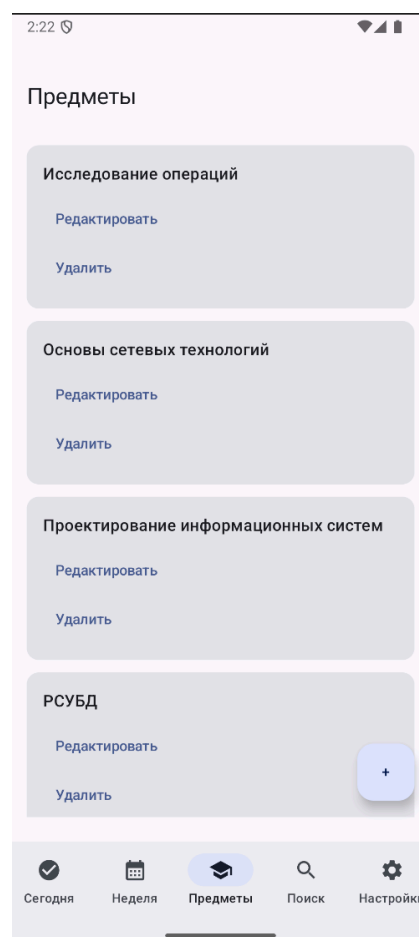


Рисунок 3.4 — Экран списка учебных предметов

После выбора предмета пользователь переходит на экран задач выбранного предмета. На данном экране отображаются только те задачи, которые относятся к конкретному учебному предмету. Это позволяет удобно организовать учебную деятельность и разделять задания по дисциплинам. Пользователь может создавать новые задачи, редактировать существующие или удалять их.

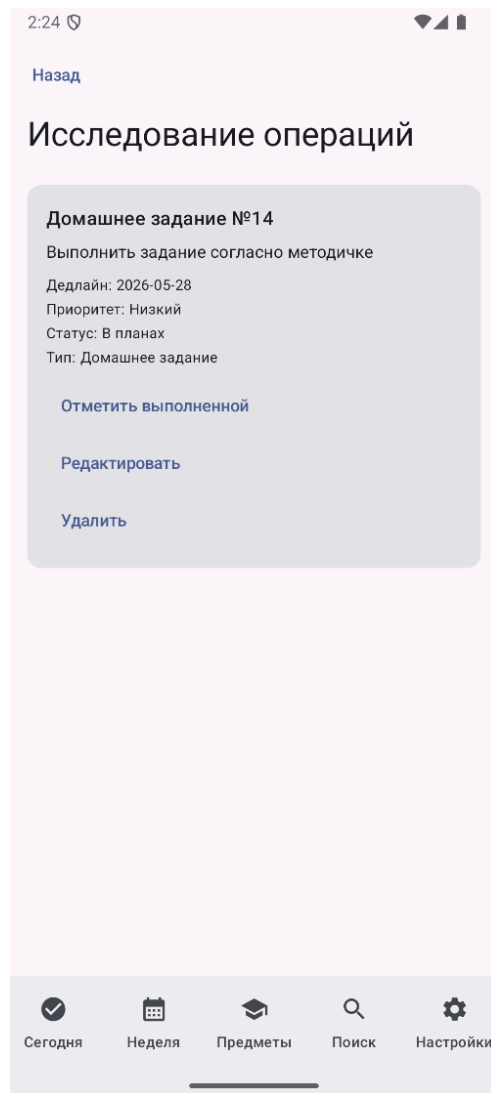


Рисунок 3.5 — Экран задач выбранного предмета

В приложении также реализован экран задач на неделю. Данный экран предназначен для отображения задач, сроки выполнения которых приходятся на ближайшие семь дней. Такой подход позволяет пользователю быстро оценить текущую учебную нагрузку и своевременно планировать выполнение заданий. Задачи отображаются в хронологическом порядке, что упрощает контроль дедлайнов.

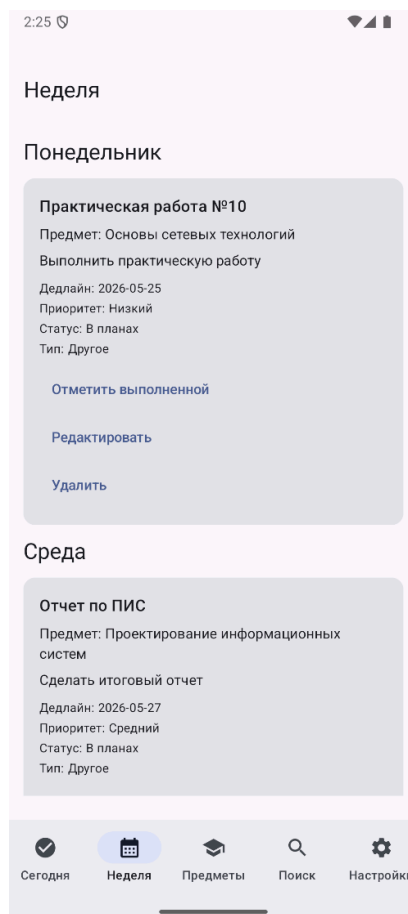


Рисунок 3.6 — Экран задач на неделю

Для быстрого поиска нужных задач в приложении реализован экран поиска. Пользователь может вводить текст поискового запроса, после чего отображаются подходящие результаты. В поисковой строке предусмотрена кнопка очистки текста, а также отображение подсказки при пустом поле ввода. При отсутствии результатов отображается специальный плейсхолдер. Также реализовано сохранение истории поиска пользователя.

Во время выполнения поискового запроса отображается индикатор загрузки `ProgressBar`. В случае возникновения ошибки сети пользователю отображается сообщение с кнопкой повторной отправки запроса.

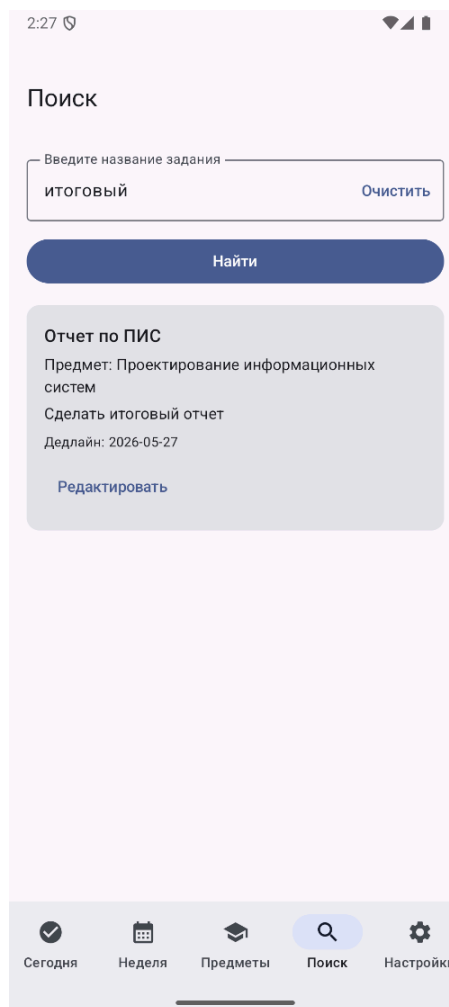


Рисунок 3.7 — Экран поиска задач

Для настройки параметров приложения был реализован экран настроек. На данном экране пользователь может изменить тему оформления приложения, включая переключение между светлой и темной темой интерфейса. Использование темной темы позволяет повысить комфорт работы пользователя при слабом освещении.

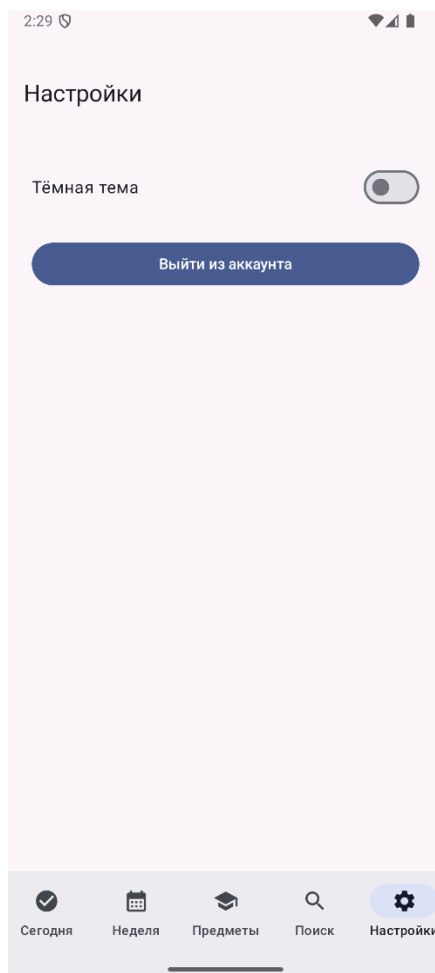


Рисунок 3.8 — Экран настроек приложения

Навигация между экранами приложения реализована с использованием `Navigation Compose`. Данный инструмент обеспечивает переход между экранами с сохранением состояния интерфейса и передачей необходимых параметров. Навигационная структура приложения включает переходы между экранами авторизации, регистрации, задач, предметов, поиска и настроек.

Для управления состоянием экранов использовались `ViewModel` и `StateFlow`. `ViewModel` отвечает за хранение данных интерфейса и обработку действий пользователя, а `StateFlow` обеспечивает реактивное обновление состояния экранов при изменении данных. Такой подход позволил разделить пользовательский интерфейс и бизнес-логику приложения.

Пример реализации `ViewModel` представлен ниже.

Листинг 3.1 — Пример реализации `ViewModel` клиентской части для задач на неделю

```

@HiltViewModel
class WeekViewModel @Inject constructor(
    getTasksUseCase: GetTasksUseCase,
    private val updateTaskStatusUseCase: UpdateTaskStatusUseCase,
    private val deleteTaskUseCase: DeleteTaskUseCase,
    private val refreshTasksUseCase: RefreshTasksUseCase
) : ViewModel() {

    private val localState = MutableStateFlow(WeekUiState())

    val uiState = combine(
        localState,
        getTasksUseCase()
    ) { state, tasks ->
        val today = LocalDate.now()
        val startOfWeek = today.with(DayOfWeek.MONDAY)
        val endOfWeek = today.with(DayOfWeek.SUNDAY)

        val activeTasks = tasks.filter { it.status !=
TaskStatus.COMPLETED }

        val weekTasks = activeTasks.filter { task ->
            task.deadline?.let { deadline ->
                runCatching {
                    val date = LocalDate.parse(deadline)
                    !date.isBefore(startOfWeek)      &&
!date.isAfter(endOfWeek)
                }.getOrElse(false)
            } == true
        }

        val tasksByDay = weekTasks.groupBy { task ->
            val date = LocalDate.parse(task.deadline)
            dayName(date.dayOfWeek)
        }

        val withoutDeadline = activeTasks.filter { it.deadline == null }

        state.copy(
            tasksByDay = tasksByDay,
            tasksWithoutDeadline = withoutDeadline
        )
    }.stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(5_000),
        initialValue = WeekUiState()
    )

    init {
        refreshTasks()
    }

    fun refreshTasks() {
        viewModelScope.launch {
            localState.value = localState.value.copy(
                isLoading = true,

```

```

        errorMessage = null
    )

    runCatching {
        refreshTasksUseCase()
    }.onFailure { error ->
        localState.value = localState.value.copy(
            errorMessage = error.message ?: "Не удалось загрузить
задачи"
        )
    }

    localState.value = localState.value.copy(isLoading = false)
}

private fun dayName(dayOfWeek: DayOfWeek): String {
    return when (dayOfWeek) {
        DayOfWeek.MONDAY -> "Понедельник"
        DayOfWeek.TUESDAY -> "Вторник"
        DayOfWeek.WEDNESDAY -> "Среда"
        DayOfWeek.THURSDAY -> "Четверг"
        DayOfWeek.FRIDAY -> "Пятница"
        DayOfWeek.SATURDAY -> "Суббота"
        DayOfWeek.SUNDAY -> "Воскресенье"
    }
}

fun markTaskCompleted(taskId: String) {
    viewModelScope.launch {
        updateTaskStatusUseCase(
            id = taskId,
            status = TaskStatus.COMPLETED
        )

        refreshTasks()
    }
}

fun deleteTask(taskId: String) {
    viewModelScope.launch {
        deleteTaskUseCase(taskId)
        refreshTasks()
    }
}
}

```

Для взаимодействия клиентской части с сервером использовалась библиотека Retrofit. С помощью Retrofit были реализованы HTTP-запросы к серверному REST API, включая авторизацию пользователя, получение списка

предметов и задач, создание новых записей, редактирование и удаление данных. Передача данных между клиентом и сервером осуществлялась в формате JSON.

Для хранения токена авторизации использовался механизм локального хранения данных, что позволяло сохранять авторизацию пользователя между запусками приложения. При выполнении запросов токен автоматически добавлялся в заголовок Authorization.

Для организации клиентской части приложения использовалась многослойная архитектура, основанная на подходах MVVM и Clean Architecture. Данная архитектура обеспечивает разделение пользовательского интерфейса, бизнес-логики и работы с данными, что упрощает сопровождение и расширение приложения.

Пользовательские действия обрабатываются интерфейсом, реализованным с использованием Jetpack Compose, после чего передаются в ViewModel. ViewModel отвечает за управление состоянием экранов и взаимодействие с бизнес-логикой приложения через UseCase. Получение и сохранение данных выполняется через Repository, который взаимодействует с серверной частью при помощи Retrofit API. Серверная часть приложения реализована на основе Ktor и обеспечивает обработку HTTP-запросов, авторизацию пользователей и работу с базой данных.

Архитектура клиентской части мобильного приложения представлена на рисунке 3.9.

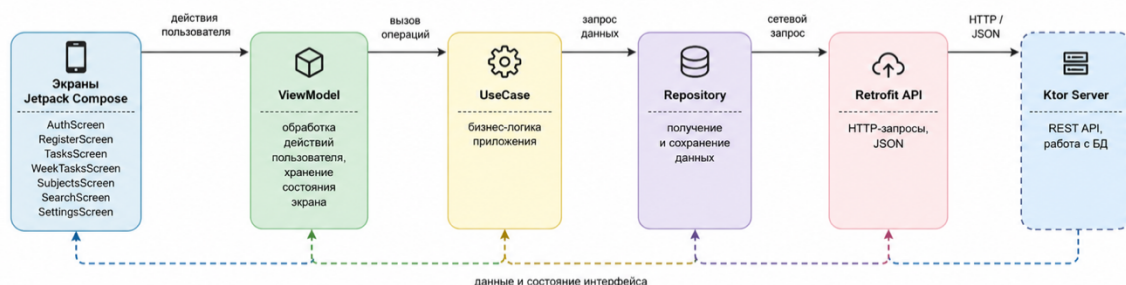


Рисунок 3.9 — Архитектура клиентской части мобильного приложения

3.2 Разработка серверной части мобильного приложения

Серверная часть мобильного приложения была разработана с использованием фреймворка Ktor на языке Kotlin. Основной задачей серверной части является обработка запросов от клиентского приложения, выполнение бизнес-логики, работа с базой данных и обеспечение авторизации пользователей.

Серверное приложение реализовано по архитектуре REST API. Взаимодействие между клиентом и сервером осуществляется посредством HTTP-запросов, а передача данных выполняется в формате JSON. Такой подход обеспечивает независимость клиентской и серверной частей приложения, а также упрощает обмен данными между ними.

Серверная часть приложения обеспечивает выполнение следующих функций:

- регистрация новых пользователей;
- авторизация пользователей;
- проверка JWT-токенов;
- создание, редактирование и удаление учебных предметов;
- создание, редактирование и удаление задач;
- получение списка задач и предметов;
- выполнение поиска задач.

Для организации серверной части использовались маршруты (routes), сервисы (services), репозитории (repositories) и модели данных. Такой подход позволяет разделить обработку HTTP-запросов, бизнес-логику и работу с базой данных.

В серверной части были реализованы отдельные маршруты для работы с авторизацией, предметами и задачами. Каждый маршрут отвечает за обработку определенного типа запросов и возврат соответствующего результата клиентскому приложению.

На сервере также реализована JWT-авторизация. После успешного входа пользователя сервер генерирует JWT-токен, который в дальнейшем используется для подтверждения личности пользователя при выполнении запросов к защищенным маршрутам. Это позволяет ограничить доступ к данным и обеспечить безопасность приложения. Для хранения данных использовалась реляционная база данных PostgreSQL.

Для организации серверной части мобильного приложения использовалась многослойная архитектура. Данная архитектура обеспечивает разделение обработки HTTP-запросов, бизнес-логики и взаимодействия с базой данных. Это упрощает сопровождение серверной части приложения и позволяет независимо изменять отдельные компоненты системы.

Клиентское приложение отправляет HTTP-запросы на сервер в формате JSON. Серверная часть, реализованная на основе Ktor, принимает запросы через REST API и передает их в маршруты обработки запросов (Routes). Далее выполняется бизнес-логика приложения, включающая авторизацию пользователей, обработку данных и выполнение CRUD-операций. Для взаимодействия с базой данных используется слой Repository (DAO), обеспечивающий выполнение операций чтения и изменения данных в PostgreSQL.

Архитектура серверной части мобильного приложения представлена на рисунке 3.10.

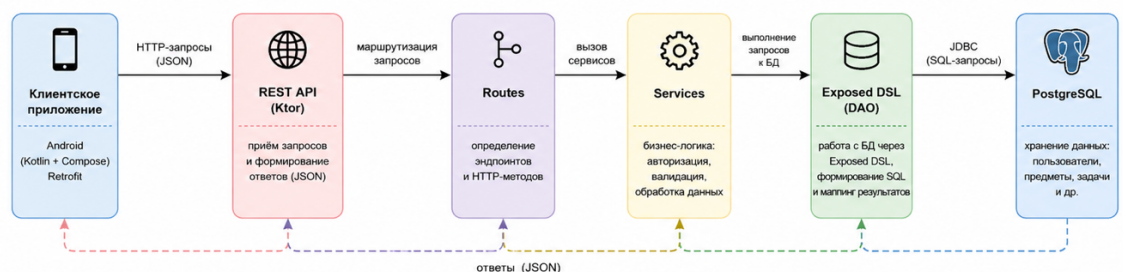


Рисунок 3.10 — Архитектура серверной части мобильного приложения

3.2.1 Описание основной логики сервера

Основная логика серверной части мобильного приложения заключается в обработке запросов, поступающих от клиентского приложения, выполнении проверки данных, авторизации пользователей и взаимодействии с базой данных. Серверная часть реализована по принципу REST API и обеспечивает обмен данными между клиентом и сервером в формате JSON.

После получения HTTP-запроса сервер определяет необходимый маршрут обработки запроса. В зависимости от типа запроса выполняются операции регистрации пользователя, авторизации, получения данных, создания новых записей, изменения существующих данных или удаления информации из базы данных.

Для авторизации пользователей на сервере реализован механизм JWT-аутентификации. После успешного входа пользователя сервер генерирует JWT-токен, содержащий информацию о пользователе. Данный токен передается клиентскому приложению и используется при выполнении последующих запросов к защищенным маршрутам сервера.

При обработке запросов сервер выполняет проверку корректности входных данных. Например, проверяется наличие обязательных полей, корректность идентификаторов, а также принадлежность данных конкретному пользователю. Это позволяет предотвратить некорректную работу приложения и ограничить доступ к данным других пользователей.

В серверной части реализованы CRUD-операции для работы с учебными предметами и задачами. Пользователь может создавать новые записи, просматривать существующие данные, изменять их и удалять ненужную информацию. Все операции выполняются только для авторизованного пользователя.

Пример реализации маршрута получения списка задач представлен ниже.

Листинг 3.2 — Пример реализации маршрута получения задач на сегодня

```
route("/api/tasks") {  
  get("/today") {
```

```

        val userSession = call.getCurrentUserSession()
        val today = LocalDate.now().toString()
        val tasks = getTasksForTodayUseCase(
            firebaseUid = userSession.firebaseUid,
            today = today
        )
        call.respond(tasks.map { it.toResponse() })
    }
}

```

Для хранения информации используются сущности базы данных, связанные с таблицами PostgreSQL.

Взаимодействие между слоями серверной части организовано следующим образом: маршруты принимают HTTP-запросы, бизнес-логика выполняет обработку данных, после чего через Repository осуществляется обращение к базе данных PostgreSQL. Полученные результаты преобразуются в JSON и отправляются обратно клиентскому приложению.

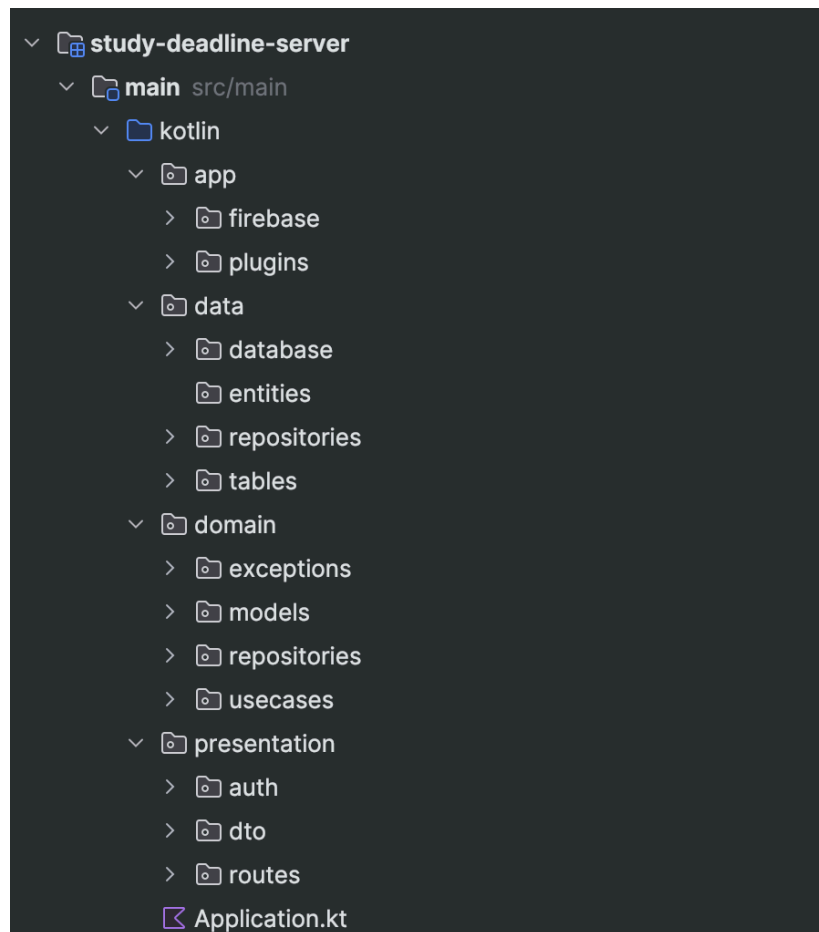


Рисунок 3.11 — Структура серверной части приложения

3.2.2 Подключение базы данных к серверу

Для хранения данных мобильного приложения была использована реляционная система управления базами данных PostgreSQL. В базе данных хранилась информация о пользователях, учебных предметах и задачах. Использование PostgreSQL позволило обеспечить надежное хранение данных и поддержку выполнения SQL-операций.

Подключение серверной части к базе данных было реализовано с использованием библиотеки Exposed DSL и технологии JDBC. JDBC обеспечивал установление соединения между сервером и PostgreSQL, а Exposed DSL использовался для выполнения запросов к базе данных средствами Kotlin.

Использование Exposed DSL позволило работать с таблицами базы данных через Kotlin-код без необходимости написания большого количества SQL-запросов вручную. Для каждой таблицы были созданы соответствующие объекты и модели данных, описывающие структуру таблиц и их поля.

В серверной части были реализованы таблицы пользователей, учебных предметов и задач. Каждая таблица содержала необходимые поля, включая идентификаторы записей, текстовые данные и связи между сущностями. Например, таблица задач содержала информацию о названии задачи, описании, дедлайне и идентификаторе пользователя, которому принадлежит задача.

Для выполнения операций чтения, добавления, изменения и удаления данных использовались DAO и запросы Exposed DSL. Это позволило реализовать CRUD-операции для работы с задачами и предметами, а также обеспечить фильтрацию данных по конкретному пользователю.

Подключение к PostgreSQL выполнялось через конфигурационные параметры сервера, включающие адрес базы данных, имя пользователя, пароль и JDBC-драйвер. После запуска сервер автоматически устанавливал соединение с базой данных и подготавливал необходимые таблицы для работы приложения.

При обработке запросов сервер обращается к таблицам базы данных через Exposed DSL, после чего полученные данные преобразуются в Kotlin-объекты и передаются обратно клиентскому приложению в формате JSON.

Пример подключения серверной части к базе данных PostgreSQL представлен ниже.

Листинг 3.3 — Подключение к базе данных PostgreSQL через JDBC и Exposed DSL

```
object DatabaseFactory {  
  
    fun init() {  
        Database.connect(hikariDataSource())  
  
        transaction {  
            SchemaUtils.create(  
                UsersTable,  
                SubjectsTable,  
                TasksTable  
            )  
        }  
        println("Database connection initialized")  
    }  
  
    private fun hikariDataSource(): HikariDataSource {  
        val config = HikariConfig()  
  
        config.jdbcUrl = System.getenv("DATABASE_URL")  
            ?: "jdbc:postgresql://localhost:5432/study_deadline"  
  
        config.username = System.getenv("DATABASE_USER")  
            ?: "postgres"  
  
        config.password = System.getenv("DATABASE_PASSWORD")  
            ?: "postgres"  
  
        config.driverClassName = "org.postgresql.Driver"  
        config.maximumPoolSize = 5  
        config.isAutoCommit = false  
        config.transactionIsolation = "TRANSACTION_REPEATABLE_READ"  
  
        return HikariDataSource(config)  
    }  
}
```

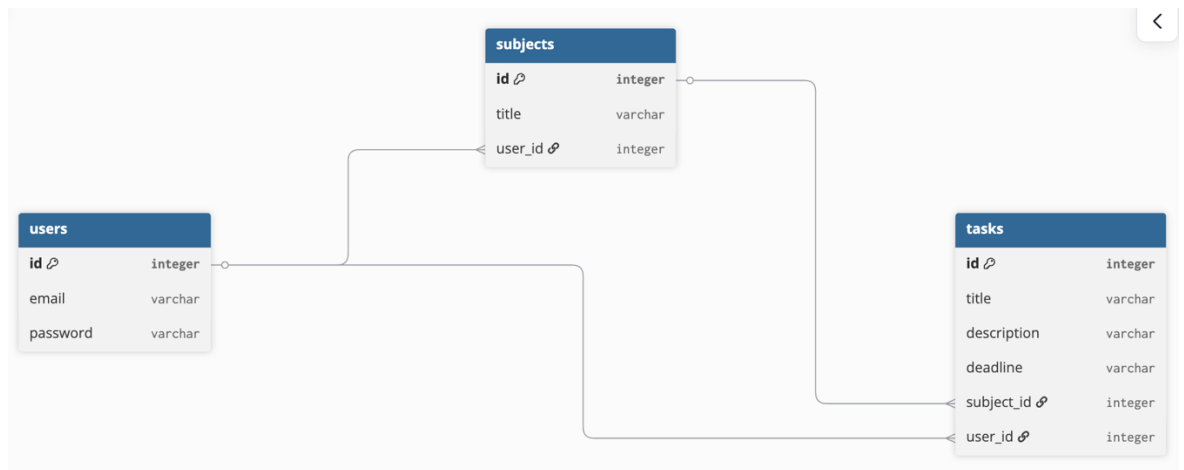


Рисунок 3.12 — Структура базы данных мобильного приложения

3.2.3 Подключение сервера к клиентской части мобильного приложения

Для обеспечения взаимодействия клиентской и серверной частей мобильного приложения использовался REST API. Обмен данными между приложением и сервером осуществлялся посредством HTTP-запросов в формате JSON. Такой подход позволил организовать независимую работу клиентской и серверной частей приложения.

На стороне клиентского приложения для выполнения сетевых запросов использовалась библиотека Retrofit. Retrofit обеспечивал отправку HTTP-запросов к серверу, получение ответов и автоматическое преобразование JSON-данных в объекты Kotlin.

Пример интерфейса взаимодействия клиентской части с сервером представлен ниже.

Листинг 3.4 — Интерфейс Retrofit API клиентской части приложения

```

interface TaskApi {

    @GET("tasks")
    suspend fun getTasks(): List<TaskDto>

    @GET("tasks/today")
    suspend fun getTodayTasks(): List<TaskDto>

    @GET("subjects/{subjectId}/tasks")
    suspend fun getTasksBySubject(
        @Path("subjectId") subjectId: String
    )
  }
  
```

```

    ): List<TaskDto>

    @POST("tasks")
    suspend fun createTask(
        @Body request: CreateTaskRequest
    ): TaskDto

    @PUT("tasks/{id}")
    suspend fun updateTask(
        @Path("id") id: String,
        @Body request: UpdateTaskRequest
    ): TaskDto

    @PATCH("tasks/{id}/status")
    suspend fun updateTaskStatus(
        @Path("id") id: String,
        @Body request: UpdateTaskStatusRequest
    ): TaskDto

    @DELETE("tasks/{id}")
    suspend fun deleteTask(
        @Path("id") id: String
    ): MessageResponse

    @GET("tasks")
    suspend fun searchTasks(
        @Query("query") query: String
    ): List<TaskDto>

    @GET("tasks/by-subject")
    suspend fun getTasksBySubjectName(
        @Query("subject") subject: String
    ): List<TaskDto>
}

```

Для работы с сервером в клиентской части был реализован API-интерфейс, содержащий методы авторизации, регистрации, получения списка задач и предметов, а также CRUD-операции для управления данными. Каждый метод API соответствовал определенному маршруту серверной части приложения.

При авторизации пользователя клиентское приложение отправляло на сервер логин и пароль пользователя. После успешной проверки данных сервер генерировал JWT-токен и отправлял его обратно клиенту. Полученный токен сохранялся локально на устройстве и использовался при выполнении последующих запросов к защищенным маршрутам сервера.

Для передачи JWT-токена использовался HTTP-заголовок `Authorization`. Это позволяло серверу идентифицировать пользователя и предоставлять доступ

только к его данным. Благодаря этому каждый пользователь работал только со своими учебными предметами и задачами.

Пример настройки HTTP-клиента и подключения Retrofit представлен ниже.

Листинг 3.5 — Настройка Retrofit и HTTP-клиента

```
@Provides
@Singleton
fun provideOkHttpClient(
    authInterceptor: AuthInterceptor
): OkHttpClient {

    val loggingInterceptor = HttpLoggingInterceptor().apply {
        level = HttpLoggingInterceptor.Level.BODY
    }

    return OkHttpClient.Builder()
        .addInterceptor(authInterceptor)
        .addInterceptor(loggingInterceptor)
        .build()
}

@Provides
@Singleton
fun provideRetrofit(
    okHttpClient: OkHttpClient,
    gson: Gson
): Retrofit {
    return Retrofit.Builder()
        .baseUrl(BASE_URL)
        .client(okHttpClient)
        .addConverterFactory(GsonConverterFactory.create(gson))
        .build()
}
```

При выполнении сетевых запросов в приложении также реализовывалась обработка ошибок соединения и отображение индикатора загрузки. В случае возникновения ошибки пользователю отображалось соответствующее сообщение с возможностью повторной отправки запроса.

Взаимодействие между клиентом и сервером обеспечило синхронизацию данных, хранение информации в централизованной базе данных и поддержку авторизации пользователей.

3.3 Тестирование разработанного приложения

После завершения разработки мобильного приложения было выполнено тестирование основных функций клиентской и серверной частей системы. Цель тестирования заключалась в проверке корректности работы приложения, устойчивости к ошибкам и соответствии реализованного функционала требованиям курсовой работы.

В процессе тестирования проверялась работа экранов авторизации и регистрации, создание и редактирование задач, работа с учебными предметами, выполнение поиска, взаимодействие клиентской и серверной частей приложения, а также корректность обработки сетевых запросов и JWT-авторизации.

Для проверки работоспособности приложения использовалось ручное тестирование. Во время тестирования выполнялась последовательная проверка пользовательских сценариев, включая корректные и ошибочные действия пользователя. Также проверялась обработка исключительных ситуаций, связанных с отсутствием сетевого подключения и вводом некорректных данных.

В ходе тестирования было установлено, что клиентская часть корректно взаимодействует с сервером, а данные успешно сохраняются и загружаются из базы данных PostgreSQL. Все основные функции приложения работают стабильно и соответствуют поставленным требованиям.

Результаты тестирования представлены в таблице 3.1.

Таблица 3.1 — Тестовые сценарии разработанного приложения

№	Тестовый сценарий	Ожидаемый результат	Фактический результат	Статус
1	Регистрация нового пользователя с корректными данными	Пользователь успешно зарегистрирован	Регистрация выполнена успешно	Пройден
2	Авторизация с корректным логином и паролем	Пользователь переходит на главный экран	Вход выполнен, открыт главный экран	Пройден

Продолжение Таблицы 3.1

3	Авторизация с неверными данными	Отображается сообщение об ошибке	Сообщение об ошибке отображается	Пройден
4	Создание нового учебного предмета	Предмет добавляется в список	Предмет успешно добавлен	Пройден
5	Редактирование учебного предмета	Данные предмета изменяются	Изменения сохранены	Пройден
6	Удаление учебного предмета	Предмет удаляется из списка	Предмет удален	Пройден
7	Создание новой учебной задачи	Задача появляется в списке задач	Задача успешно создана	Пройден
8	Редактирование учебной задачи	Данные задачи обновляются	Изменения отображаются в списке	Пройден
9	Удаление учебной задачи	Задача удаляется из списка	Задача удалена	Пройден
10	Просмотр задач на неделю	Отображаются задачи с ближайшими дедлайнами	Задачи на неделю отображаются корректно	Пройден
11	Поиск задачи по названию	Отображаются задачи, соответствующие запросу	Поиск выполняется корректно	Пройден
12	Очистка поисковой строки	Текст запроса удаляется, результаты обновляются	Поисковая строка очищается	Пройден
13	Переключение темной темы	Интерфейс меняет тему оформления	Тема приложения изменяется	Пройден
14	Проверка доступа к данным разных пользователей	Пользователь видит только свои данные	Данные других пользователей не отображаются	Пройден
15	Обработка отсутствия подключения к серверу	Отображается сообщение об ошибке	Сообщение об ошибке отображается	Пройден

По результатам проведенного тестирования было установлено, что основные функции мобильного приложения работают корректно. Регистрация, авторизация, управление учебными предметами и задачами, поиск, просмотр задач на неделю и переключение темы выполняются без ошибок. Также была проверена корректность взаимодействия клиентской и серверной частей приложения, включая передачу JWT-токена и ограничение доступа к данным конкретного пользователя.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано клиент-серверное мобильное приложение-планировщик учебных задач. Разработанное приложение предназначено для организации учебного процесса пользователя, хранения информации об учебных предметах и задачах, а также для удобного просмотра актуальных дел на день, неделю и по выбранному предмету.

В рамках работы был проведен анализ предметной области, рассмотрены существующие аналоги, определены функциональные и нефункциональные требования к приложению. На этапе проектирования были разработаны макеты пользовательского интерфейса, диаграммы, логическая модель приложения и структура базы данных.

Также была выполнена программная реализация клиентской и серверной частей приложения. Клиентская часть разработана на языке Kotlin с использованием Jetpack Compose, серверная часть реализована с применением Ktor. Для взаимодействия клиента с сервером использовалась библиотека Retrofit, а хранение данных было организовано с помощью PostgreSQL и Neon Tech. Работа с базой данных на сервере осуществлялась через Exposed DSL и JDBC.

Поставленная цель курсовой работы была достигнута — разработано полнофункциональное клиент-серверное мобильное приложение, обеспечивающее управление учебными задачами и предметами пользователя. Приложение предоставляет возможность регистрации и авторизации пользователей, добавления и редактирования предметов и задач, поиска информации, а также хранения и синхронизации данных через серверную часть.

Все задачи, сформулированные во введении курсовой работы, были выполнены. В процессе разработки были изучены современные подходы к созданию мобильных клиент-серверных приложений, освоены технологии проектирования пользовательского интерфейса и организации серверной

архитектуры. Также были получены практические навыки работы с сетевым взаимодействием, базами данных и REST API.

В результате тестирования было установлено, что приложение корректно выполняет основные пользовательские сценарии и обеспечивает стабильную работу реализованного функционала. Разработанный программный продукт соответствует заявленным функциональным требованиям и может использоваться в качестве основы для дальнейшего развития и расширения возможностей системы.

В качестве направлений дальнейшего развития разработанного мобильного приложения можно выделить расширение функциональных возможностей системы. В дальнейшем возможно внедрение push-уведомлений о приближающихся задачах и дедлайнах, добавление календаря с визуализацией расписания, реализация совместного доступа к задачам, а также интеграция с внешними сервисами и образовательными платформами.

Кроме того, перспективным направлением является повышение производительности и безопасности приложения, оптимизация серверной части, а также расширение системы ролей пользователей. Дополнительно может быть реализована поддержка резервного копирования данных и синхронизации между несколькими устройствами пользователя.

Таким образом, в ходе выполнения курсовой работы были получены практические навыки проектирования и разработки клиент-серверных мобильных приложений, а также изучены современные технологии мобильной и серверной разработки. Полученные результаты подтверждают успешное достижение поставленной цели и выполнение всех задач курсовой работы.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Kotlin documentation : официальный сайт. – URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 27.05.2026). – Текст : электронный.
2. Android Developers. Jetpack Compose : официальный сайт. – URL: <https://developer.android.com/compose> (дата обращения: 27.05.2026). – Текст : электронный.
3. Android Developers. Android Architecture Components : официальный сайт. – URL: <https://developer.android.com/topic/architecture> (дата обращения: 27.05.2026). – Текст : электронный.
4. Ktor documentation : официальный сайт. – URL: <https://ktor.io/docs/welcome.html> (дата обращения: 27.05.2026). – Текст : электронный.
5. Retrofit documentation : официальный сайт. – URL: <https://square.github.io/retrofit/> (дата обращения: 27.05.2026). – Текст : электронный.
6. PostgreSQL Documentation : официальный сайт. – URL: <https://www.postgresql.org/docs/> (дата обращения: 27.05.2026). – Текст : электронный.
7. Exposed ORM Framework : официальный сайт. – URL: <https://www.jetbrains.com/help/exposed/home.html> (дата обращения: 27.05.2026). – Текст : электронный.
8. Figma : официальный сайт. – URL: <https://www.figma.com/> (дата обращения: 27.05.2026). – Текст : электронный.
9. Android Developers. Navigation in Jetpack Compose : официальный сайт. – URL: <https://developer.android.com/develop/ui/compose/navigation> (дата обращения: 27.05.2026). – Текст : электронный.

10. Android Developers. Testing in Android : официальный сайт. – URL: <https://developer.android.com/training/testing> (дата обращения: 27.05.2026). – Текст : электронный.
11. Pressman, R. S. Software Engineering: A Practitioner's Approach / R. S. Pressman, B. Maxim. – New York : McGraw-Hill Education, 2020. – 880 p.
12. Sommerville, I. Software Engineering / I. Sommerville. – 10th ed. – Boston : Pearson, 2021. – 816 p.
13. Калгина, И. С. Разработка мобильных приложений : учебное пособие / И. С. Калгина. – Чита : ЗабГУ, 2022. – 163 с.
14. Алпатов, А. Н. Архитектура, проектирование и разработка программных средств : учебное пособие / А. Н. Алпатов, И. Е. Рогов. – Москва : РТУ МИРЭА, 2023. – 120 с.
15. ГОСТ Р 7.0.100–2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – Москва : Стандартинформ, 2018. – Текст : непосредственный.

ПРИЛОЖЕНИЯ

Приложение А. Ссылки на репозитории проекта

Приложение Б. Скриншоты экранов мобильного приложения

Приложение В. Тестовые сценарии мобильного приложения

Приложение А. Ссылки на репозитории проекта

Исходный код клиентской и серверной частей мобильного приложения размещен в репозитории GitHub.

Ссылка на репозиторий серверной части:

<https://github.com/shylit/study-deadline-server>

Ссылка на репозиторий клиентской части:

<https://github.com/shylit/study-deadline-client>

Приложение Б. Скриншоты экранов мобильного приложения

В приложении Б представлены скриншоты основных экранов разработанного мобильного приложения для управления учебными заданиями и дедлайнами.

В приложении приведены следующие экраны:

- экран авторизации пользователя;
- экран регистрации пользователя;
- главный экран со списком задач;
- экран задач на неделю;
- экран учебных предметов;
- экран задач выбранного предмета;
- экран поиска задач;
- экран настроек приложения;
- экран редактирования предмета;
- экран добавления задачи.

12:57

Вход

Войти

Создать аккаунт

Рисунок Б.1 – Экран авторизации пользователя

12:58

Регистрация

Зарегистрироваться

Назад

Рисунок Б.2 – Экран регистрации пользователя

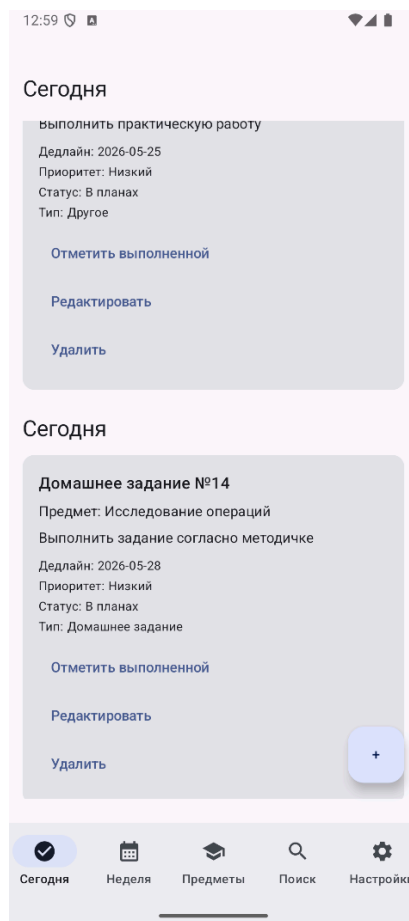


Рисунок Б.3 – Главный экран со списком задач

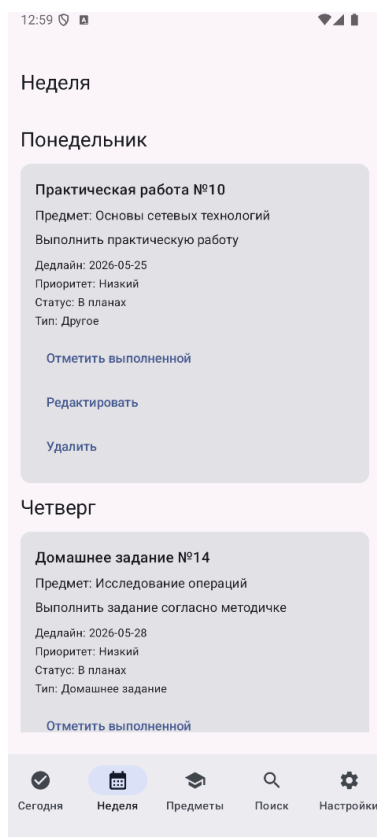


Рисунок Б.4 – Экран задач на неделю

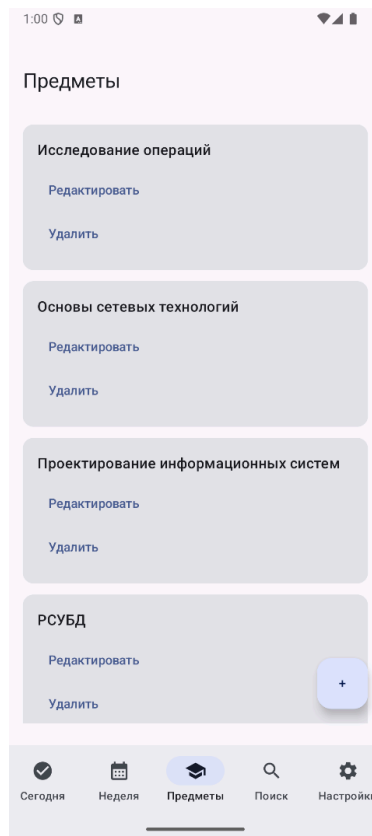


Рисунок Б.5 – Экран учебных предметов

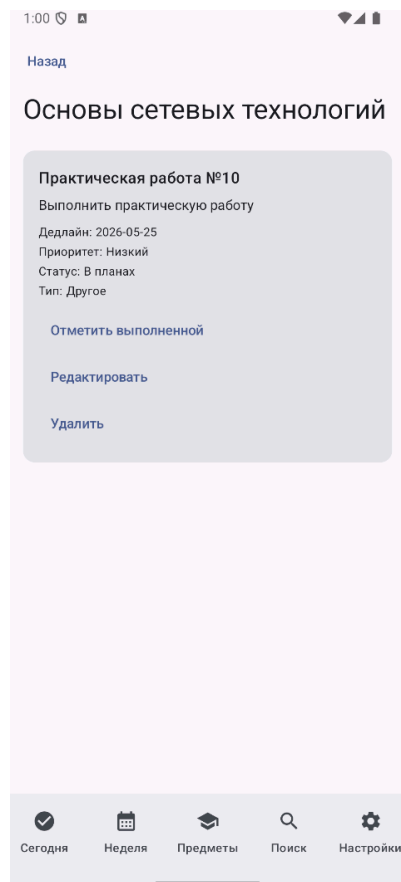


Рисунок Б.6 – Экран задач выбранного предмета

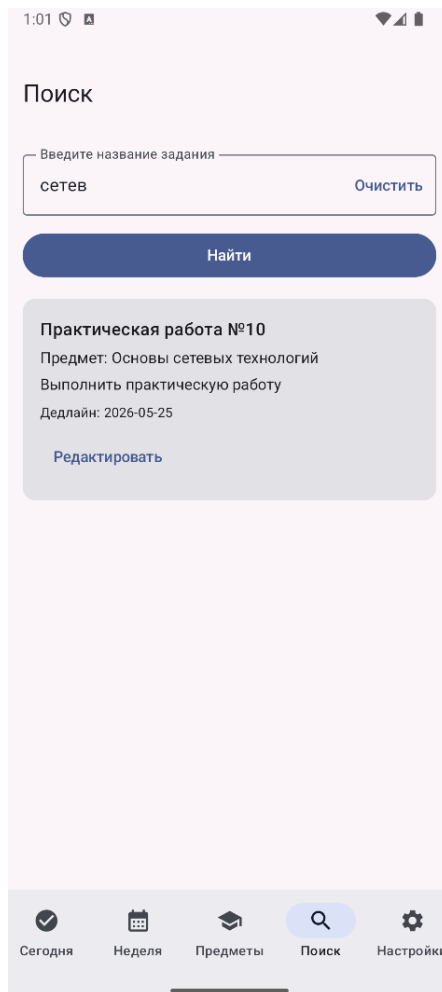


Рисунок Б.7 – Экран поиска задач

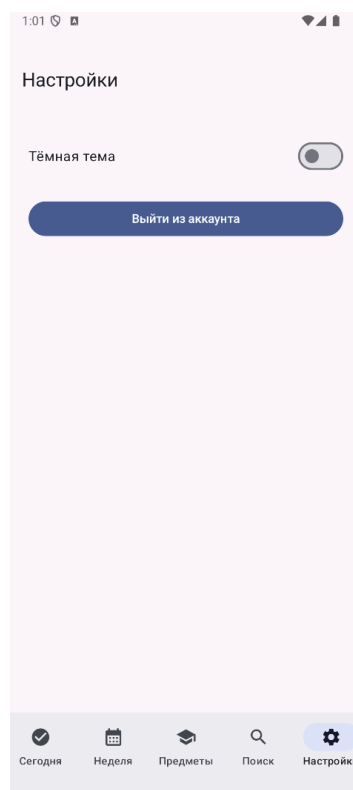


Рисунок Б.8 – Экран настроек приложения

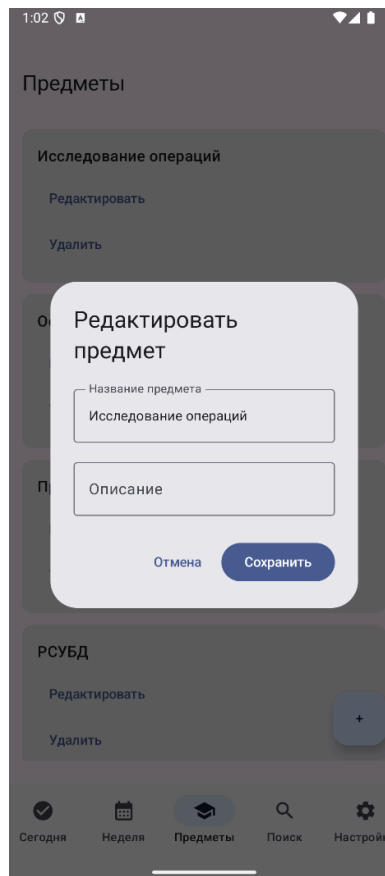


Рисунок Б.9 – Экран редактирования предмета

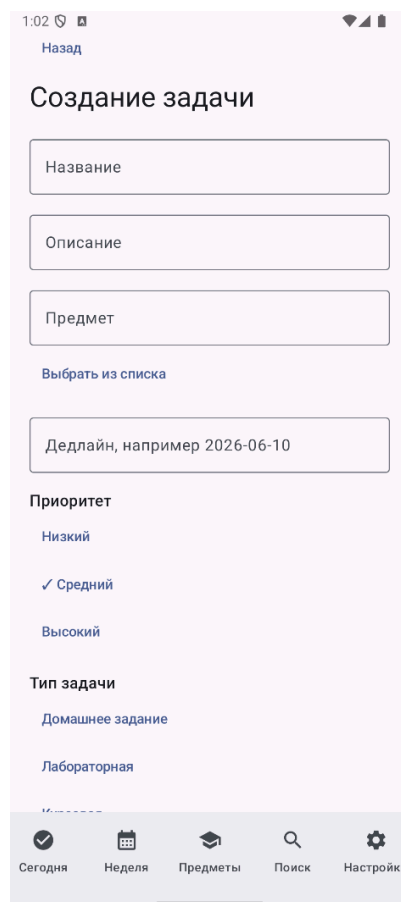


Рисунок Б.10 – Экран добавления задачи

Приложение В. Тестовые сценарии мобильного приложения

В приложении В представлены тестовые сценарии, использованные для проверки корректности работы клиентской и серверной частей мобильного приложения.

Тестирование проводилось методом ручного тестирования основных функций приложения. Проверка включала тестирование пользовательского интерфейса, работы авторизации, взаимодействия с сервером, обработки данных и выполнения CRUD-операций.

Тестовые сценарии представлены в таблице В.1.

Таблица В.1 – Тестовые сценарии мобильного приложения

№	Тестовый сценарий	Ожидаемый результат	Фактический результат	Статус
1	Регистрация нового пользователя с корректными данными	Пользователь успешно зарегистрирован	Регистрация выполнена успешно	Пройден
2	Авторизация с корректным логином и паролем	Пользователь переходит на главный экран	Вход выполнен, открыт главный экран	Пройден
3	Авторизация с неверными данными	Отображается сообщение об ошибке	Сообщение об ошибке отображается	Пройден
4	Создание нового учебного предмета	Предмет добавляется в список	Предмет успешно добавлен	Пройден
5	Редактирование учебного предмета	Данные предмета изменяются	Изменения сохранены	Пройден
6	Удаление учебного предмета	Предмет удаляется из списка	Предмет удален	Пройден
7	Создание новой учебной задачи	Задача появляется в списке задач	Задача успешно создана	Пройден
8	Редактирование учебной задачи	Данные задачи обновляются	Изменения отображаются в списке	Пройден
9	Удаление учебной задачи	Задача удаляется из списка	Задача удалена	Пройден
10	Просмотр задач на неделю	Отображаются задачи с ближайшими дедлайнами	Задачи на неделю отображаются корректно	Пройден

Продолжение Таблицы В.1

11	Поиск задачи по названию	Отображаются задачи, соответствующие запросу	Поиск выполняется корректно	Пройден
12	Очистка поисковой строки	Текст запроса удаляется, результаты обновляются	Поисковая строка очищается	Пройден
13	Переключение темной темы	Интерфейс меняет тему оформления	Тема приложения изменяется	Пройден
14	Проверка доступа к данным разных пользователей	Пользователь видит только свои данные	Данные других пользователей не отображаются	Пройден
15	Обработка отсутствия подключения к серверу	Отображается сообщение об ошибке	Сообщение об ошибке отображается	Пройден